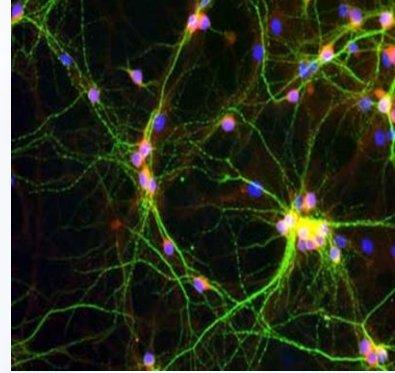


Artificial Neural Networks

Lecture 4



Plutarch: the brain is not a container needs to be filled with knowledge, but a torch needs to be lighted





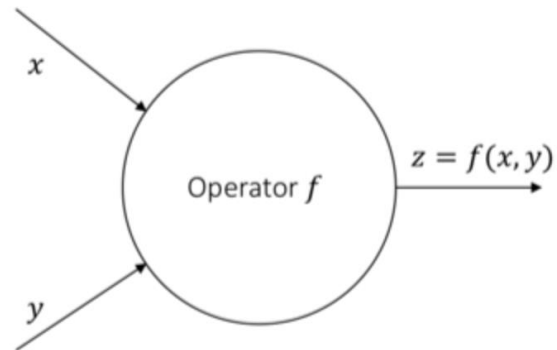
Multilayer Neural Networks

**❖ Back Propagation for
computing differentiation**



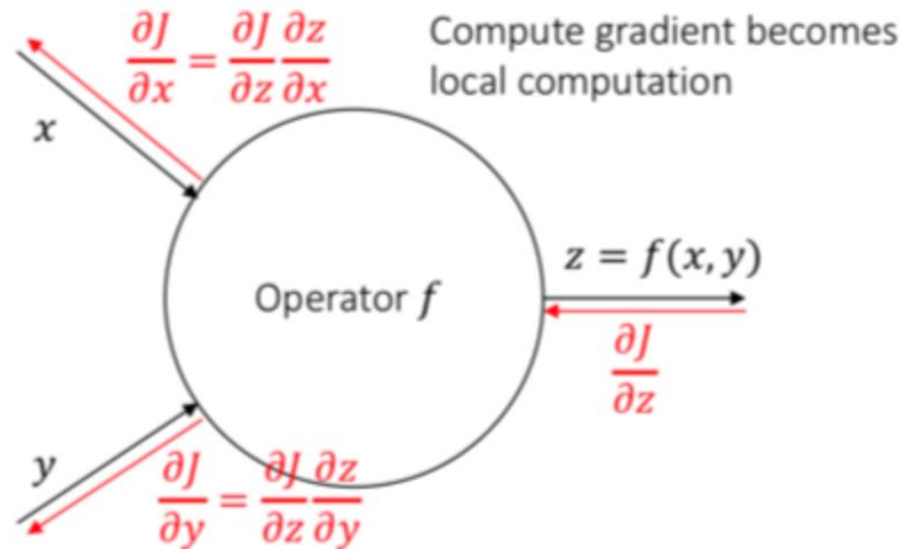
Back Propagation for computing differentiation

Backpropagation



Back Propagation for computing differentiation

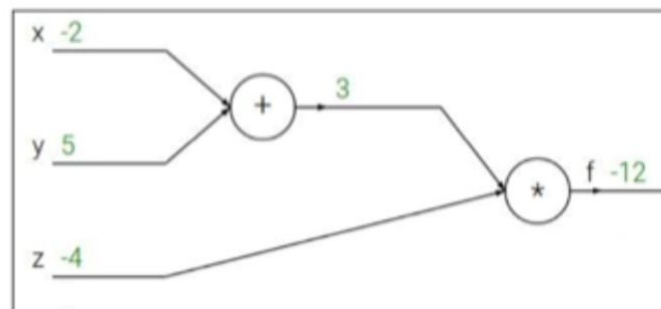
Backpropagation



Example: Back-Prop

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2$, $y = 5$, $z = -4$



Want: $\frac{\partial f}{\partial x}$, $\frac{\partial f}{\partial y}$, $\frac{\partial f}{\partial z}$

- slide source: Stanford CS231n lecture note.

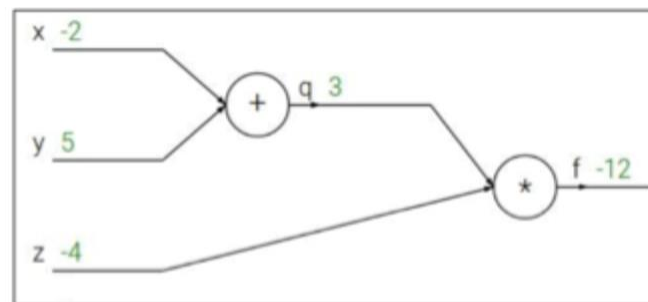
Example: Back-Prop

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$



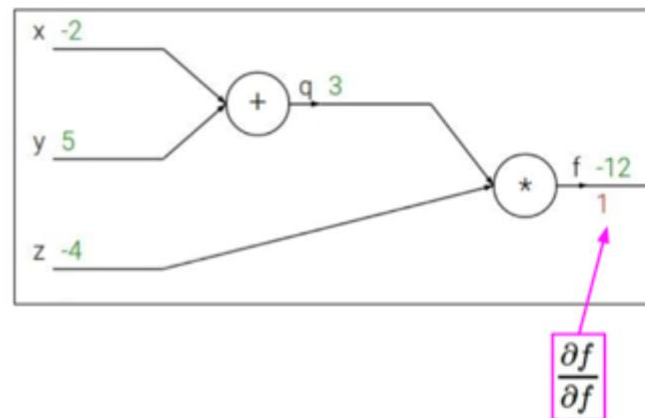
Example: Back-Prop

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$



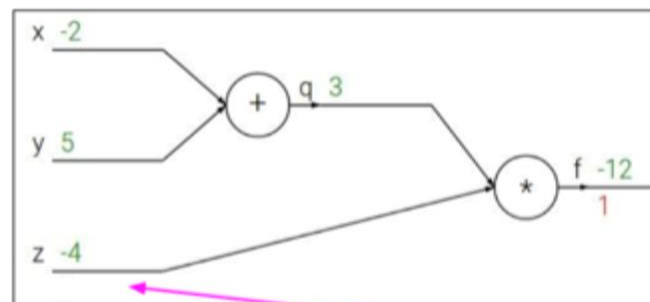
Example: Back-Prop

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$



$$\frac{\partial f}{\partial z}$$

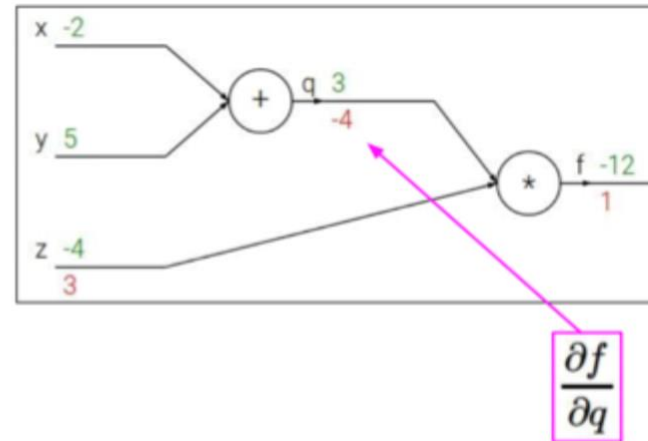
Example: Back-Prop

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$



Example: Back-Prop

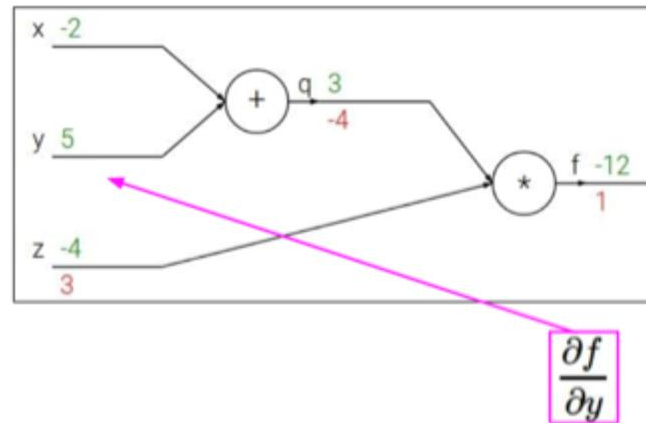
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Example: Back-Prop

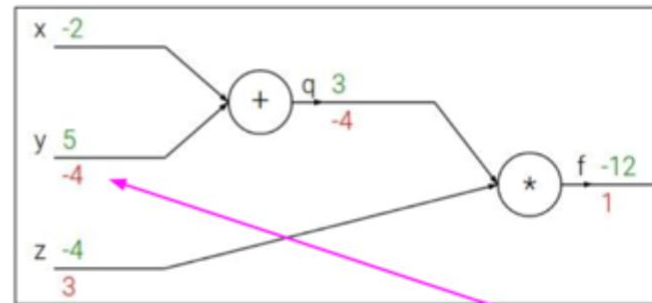
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Chain rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y}$$

Upstream Local
Gradient Gradient

$$\frac{\partial f}{\partial y}$$

Example: Back-Prop

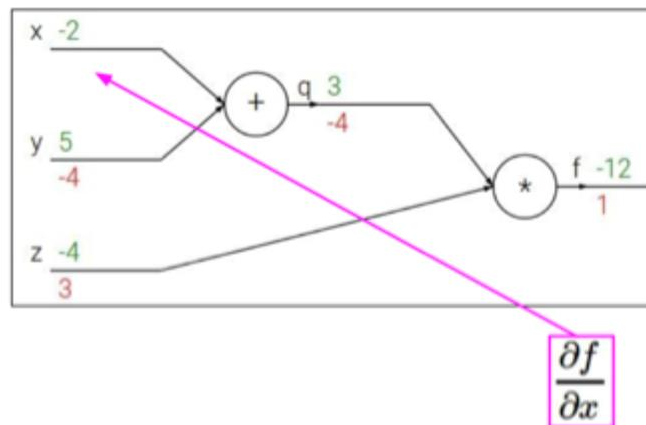
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Example: Back-Prop

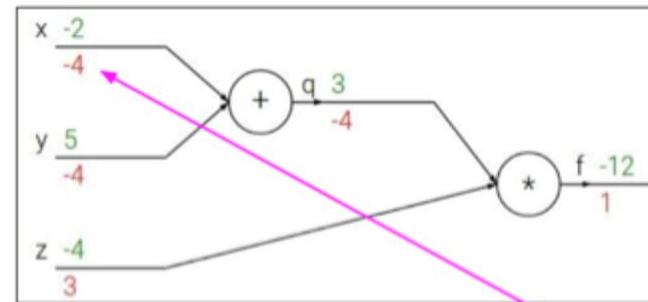
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

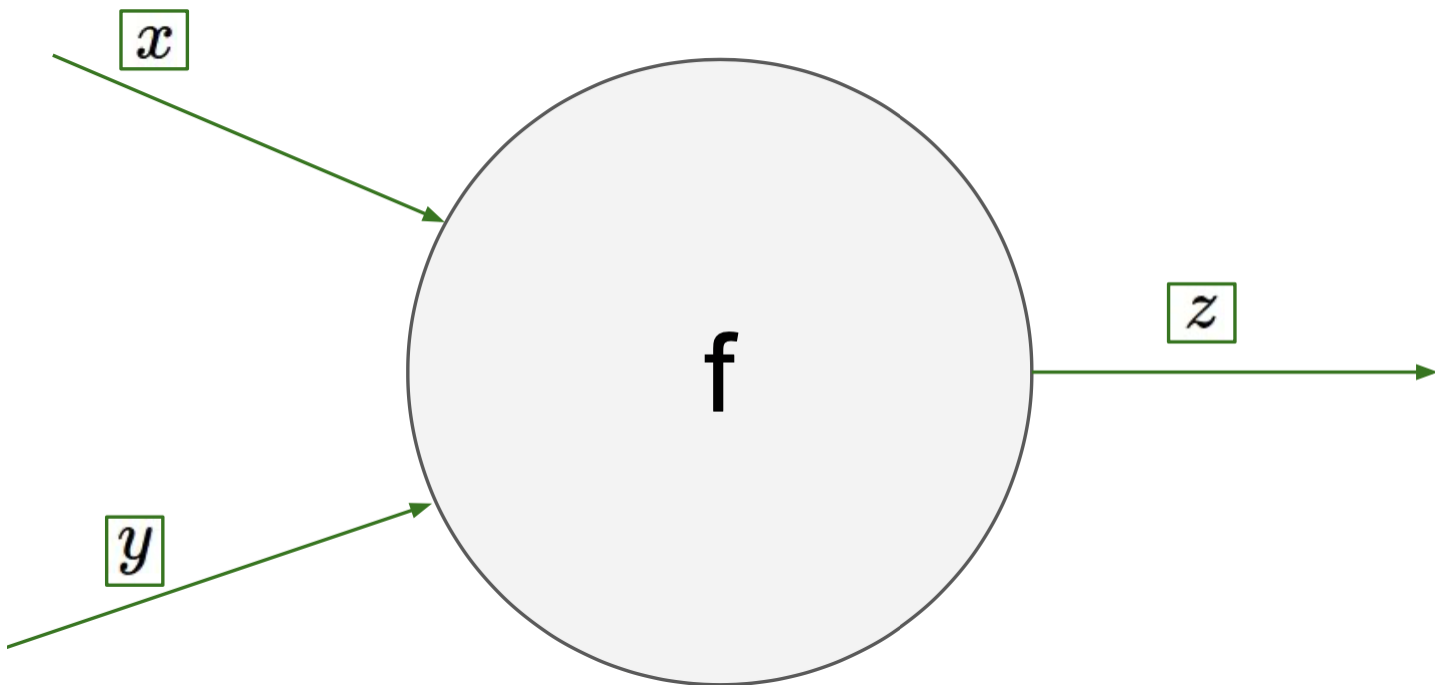
Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

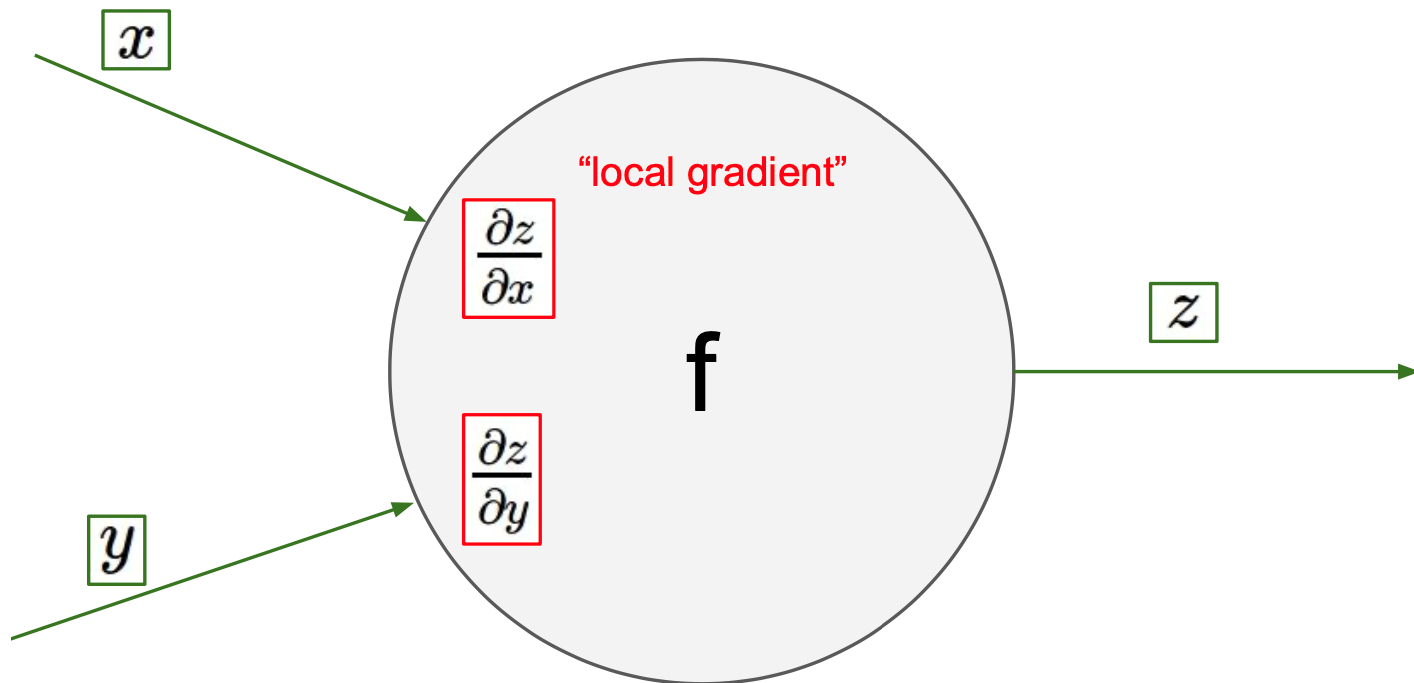


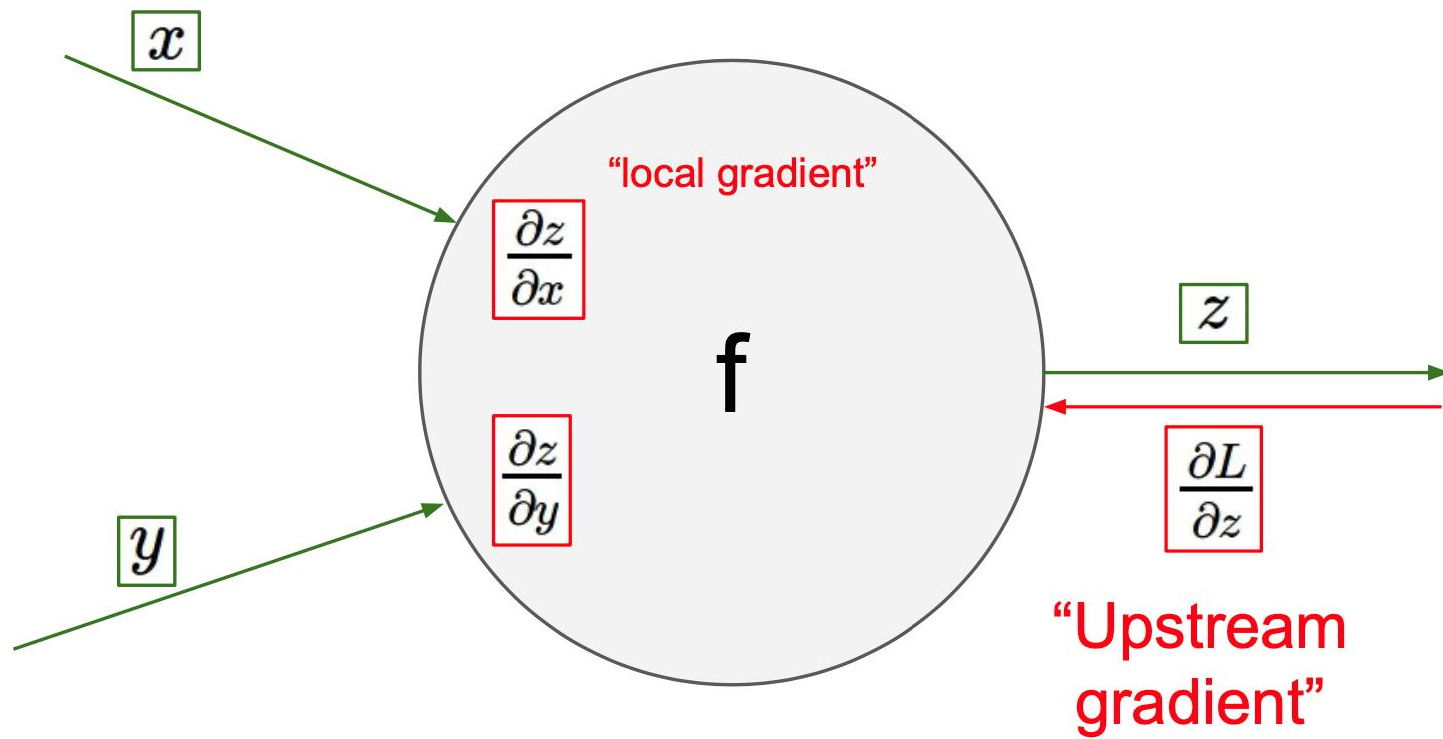
Chain rule:

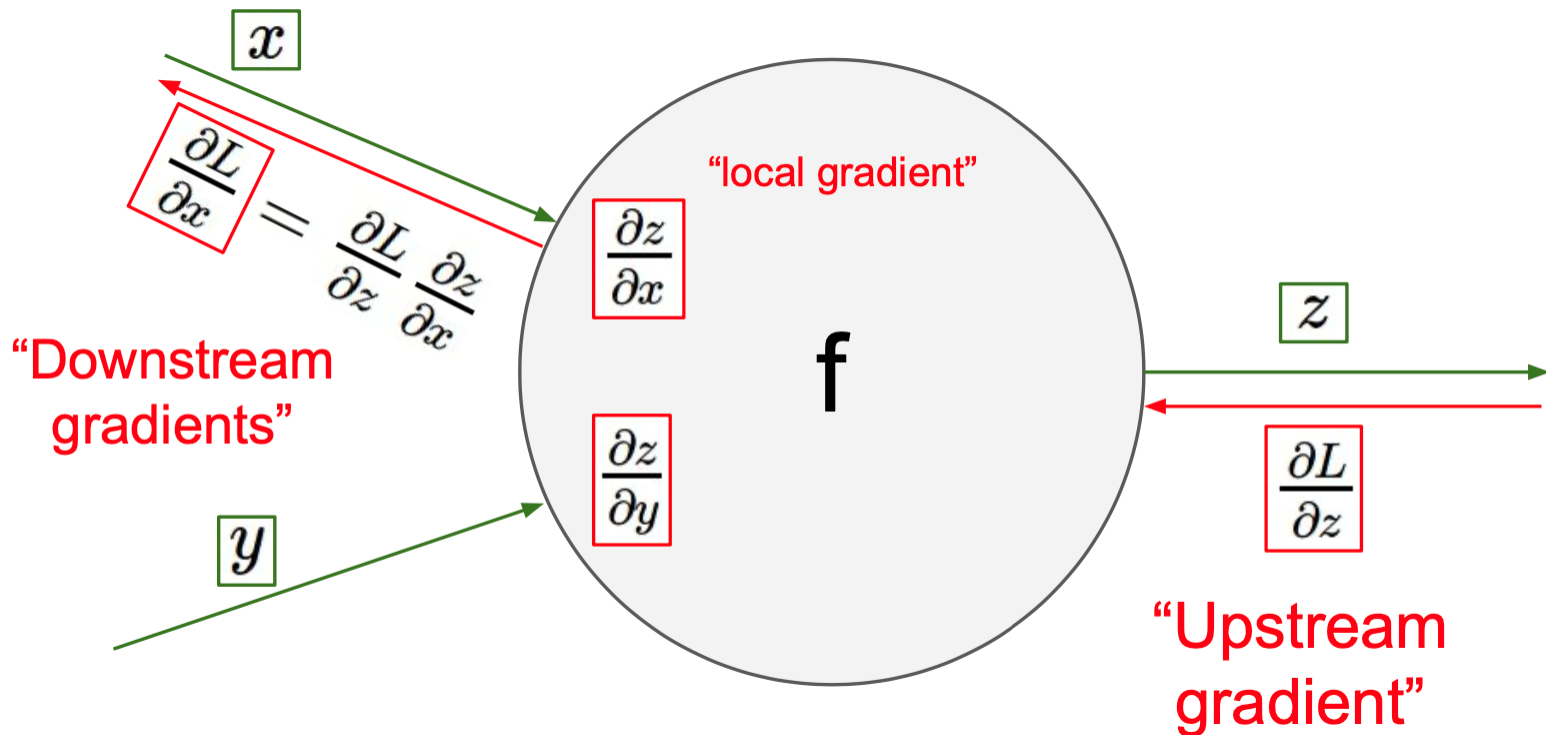
$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x}$$

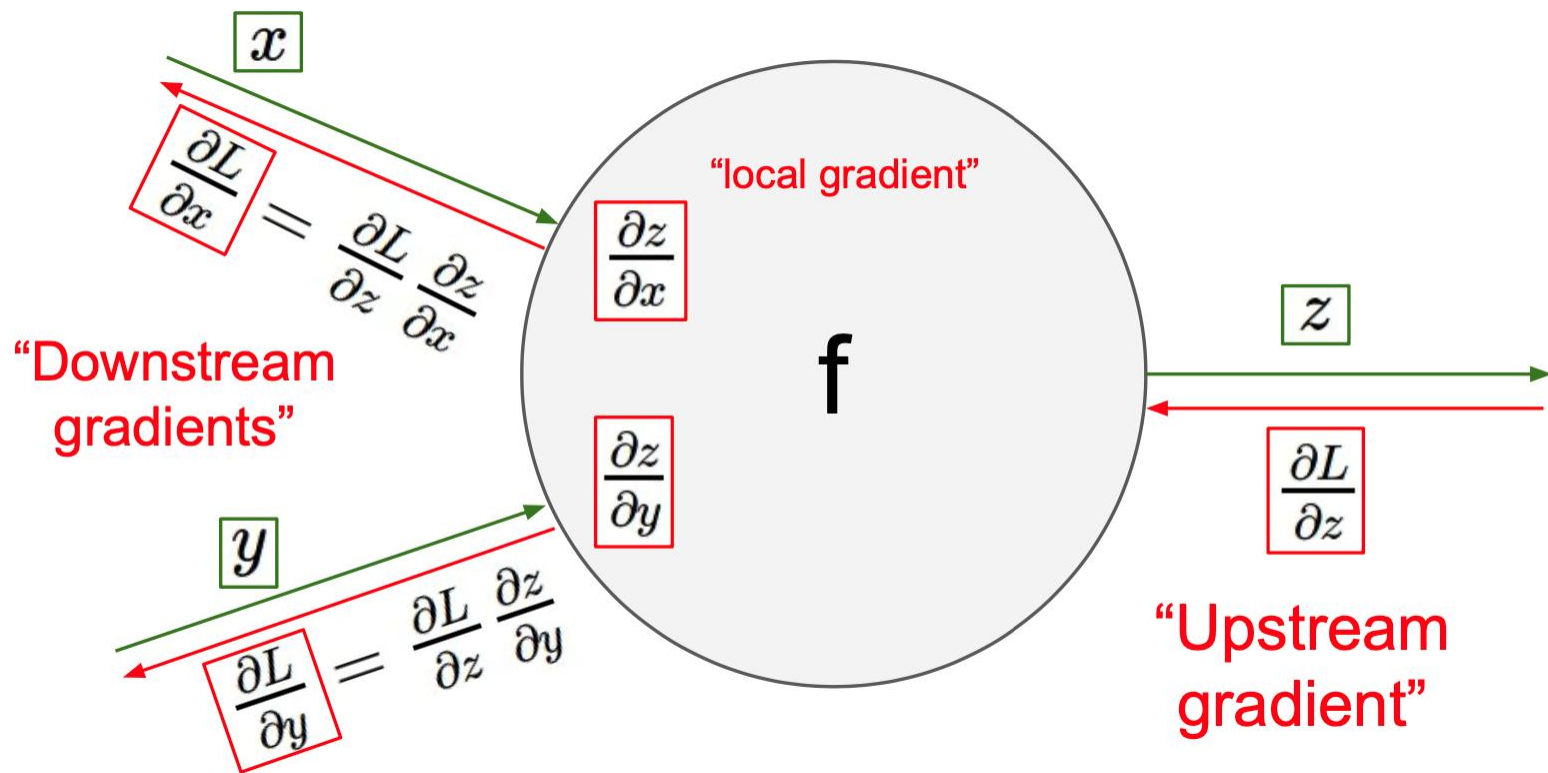
Upstream Local
Gradient Gradient

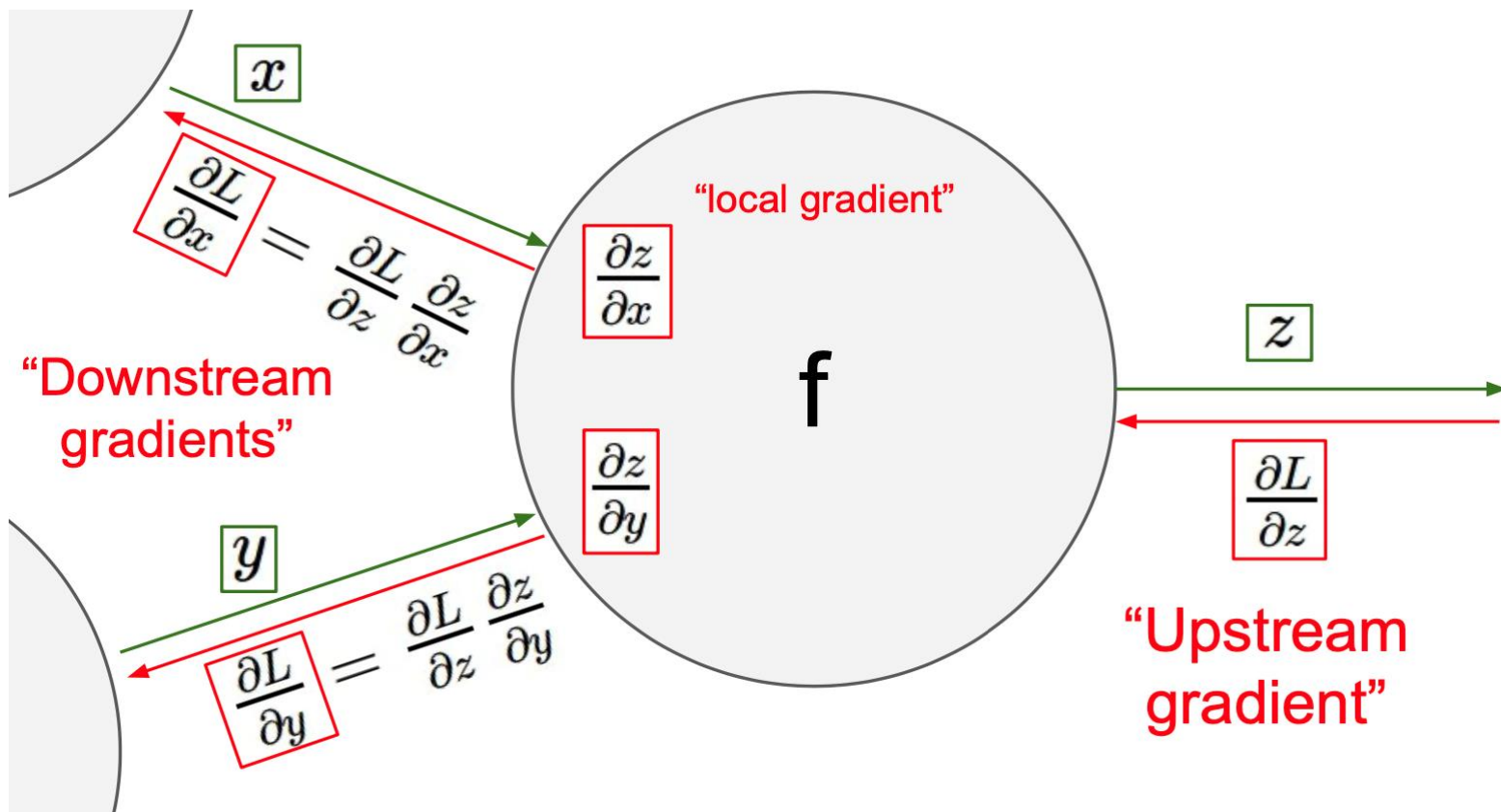




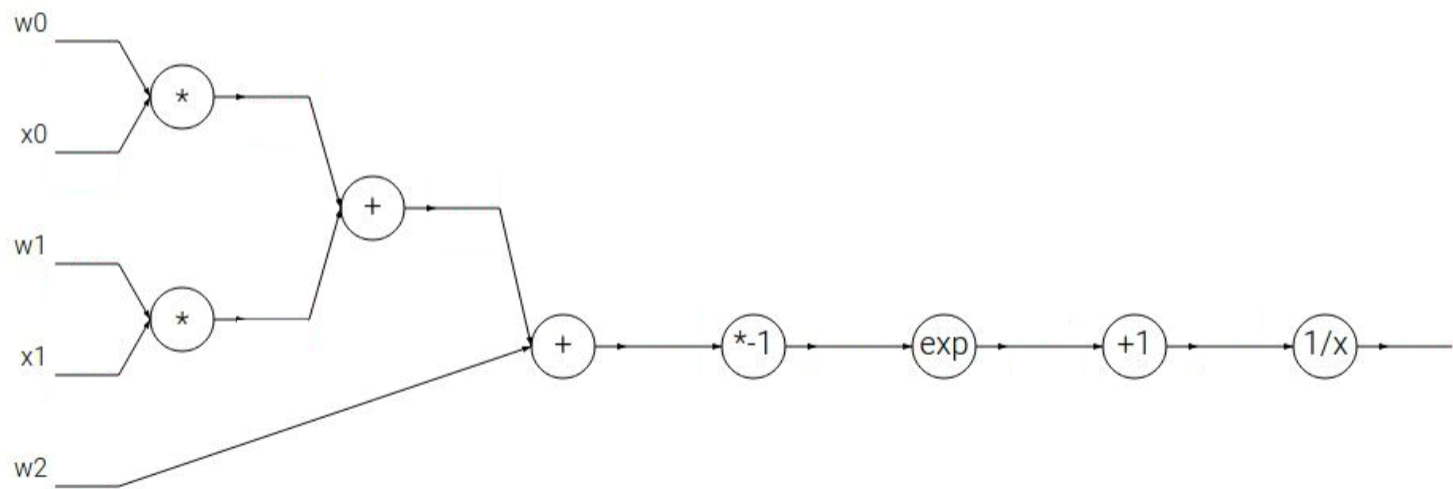




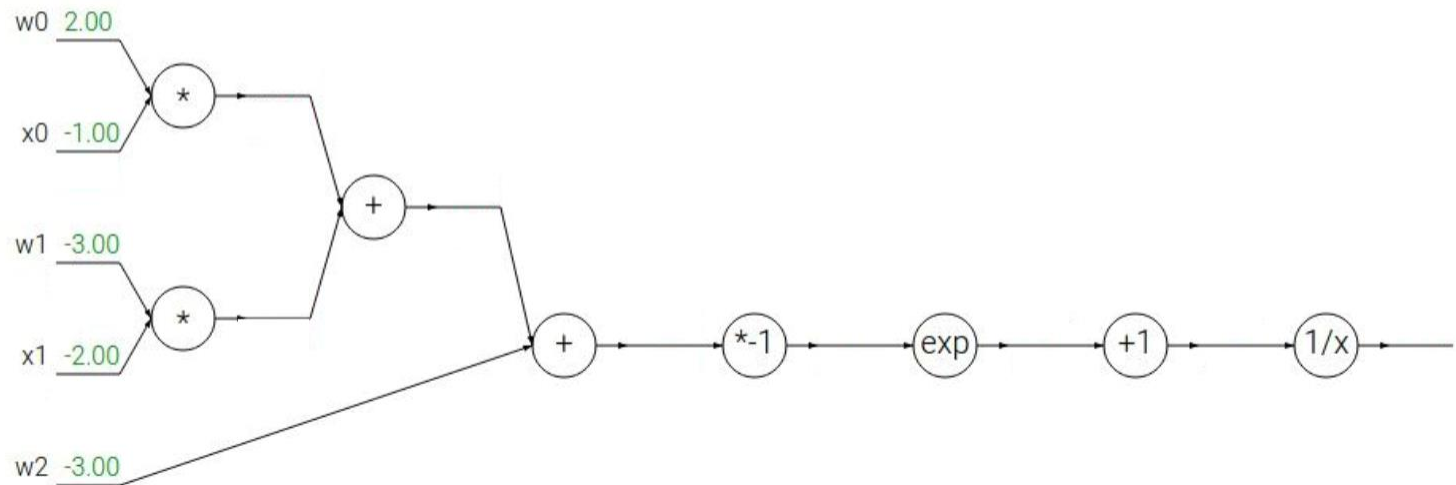




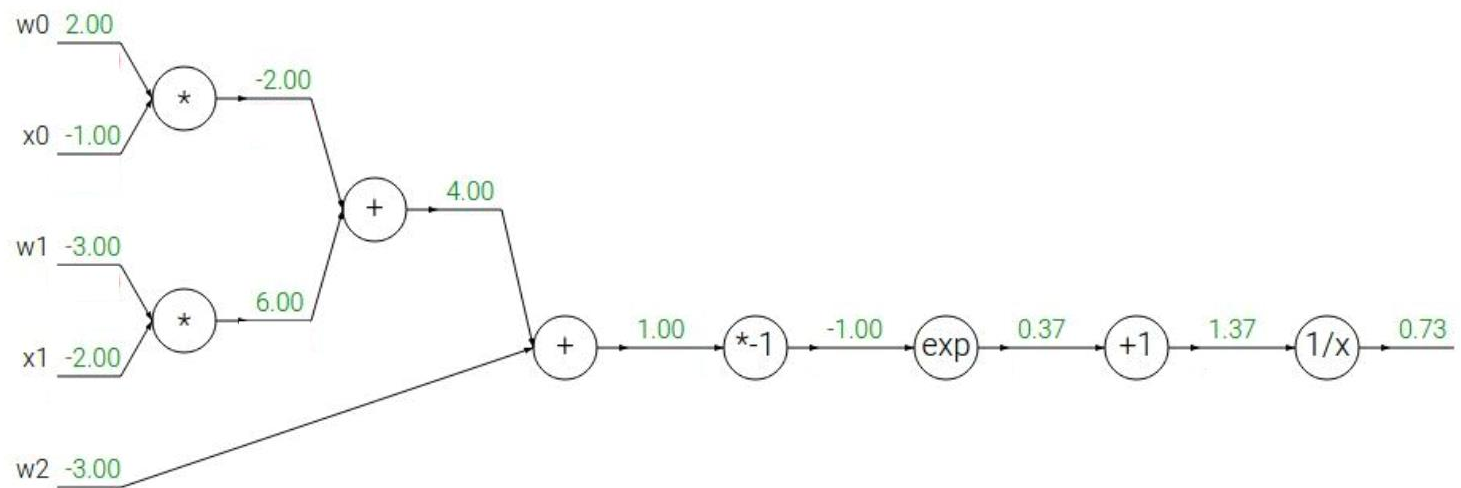
Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$



Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$



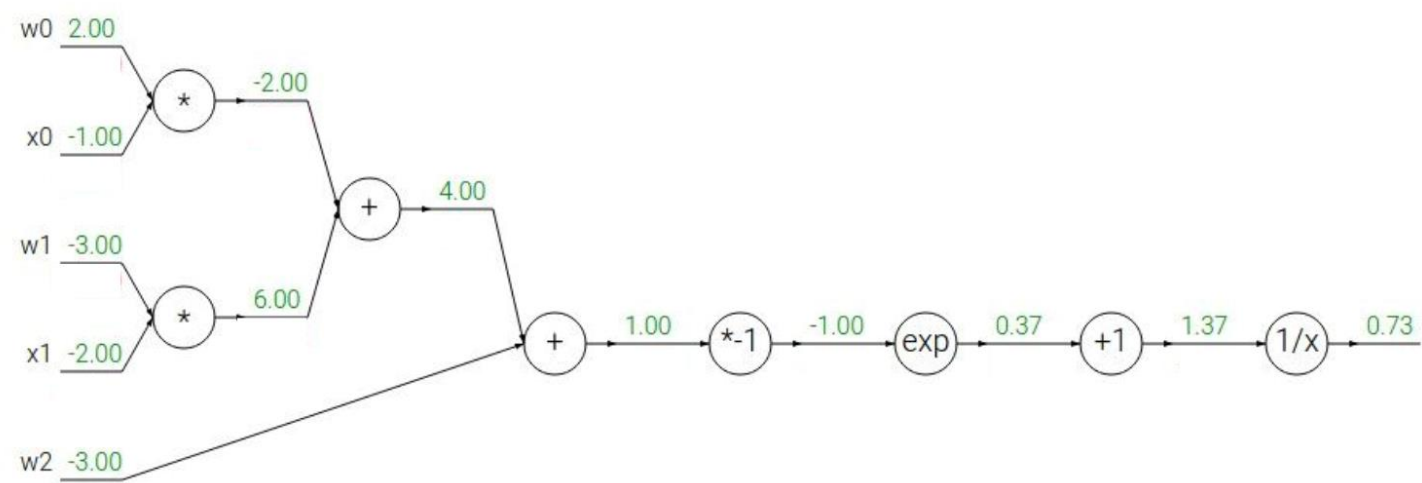
Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$





Another example:

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$

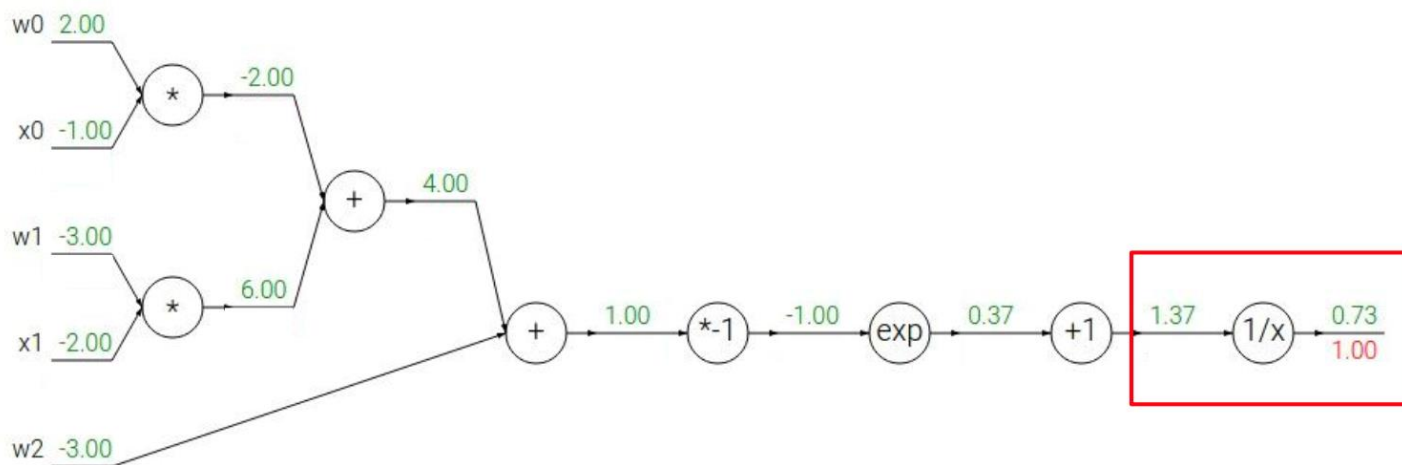


$f(x) = e^x$	→	$\frac{df}{dx} = e^x$		$f(x) = \frac{1}{x}$	→	$\frac{df}{dx} = -1/x^2$
$f_a(x) = ax$	→	$\frac{df}{dx} = a$		$f_c(x) = c + x$	→	$\frac{df}{dx} = 1$



Another example:

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$



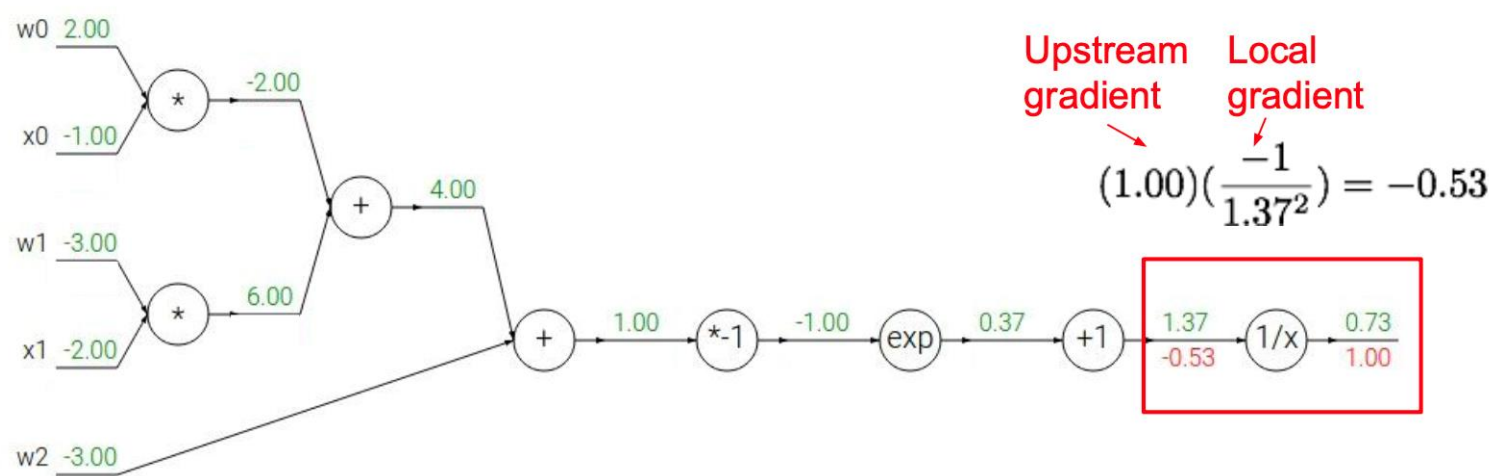
$$\begin{aligned} f(x) &= e^x & \rightarrow & \quad \frac{df}{dx} = e^x \\ f_a(x) &= ax & \rightarrow & \quad \frac{df}{dx} = a \end{aligned}$$

$$\begin{aligned} f(x) &= \frac{1}{x} & \rightarrow & \quad \frac{df}{dx} = -1/x^2 \\ f_c(x) &= c + x & \rightarrow & \quad \frac{df}{dx} = 1 \end{aligned}$$



Another example:

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$

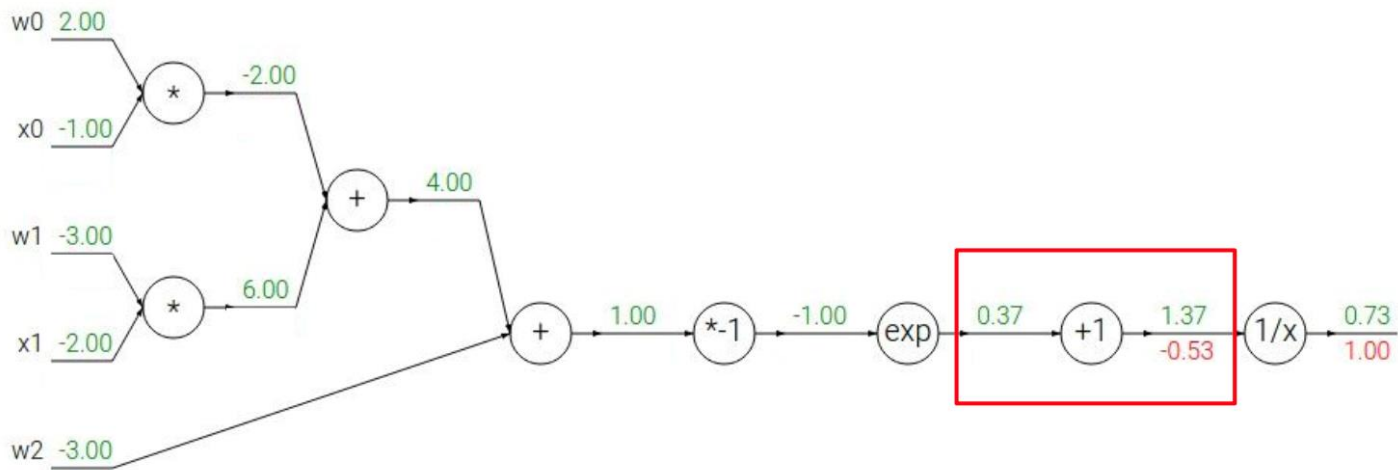


$f(x) = e^x$	$\rightarrow$	$\frac{df}{dx} = e^x$
$f_a(x) = ax$	$\rightarrow$	$\frac{df}{dx} = a$

$f(x) = \frac{1}{x}$	$\rightarrow$	$\frac{df}{dx} = -1/x^2$
$f_c(x) = c + x$	$\rightarrow$	$\frac{df}{dx} = 1$

Another example:

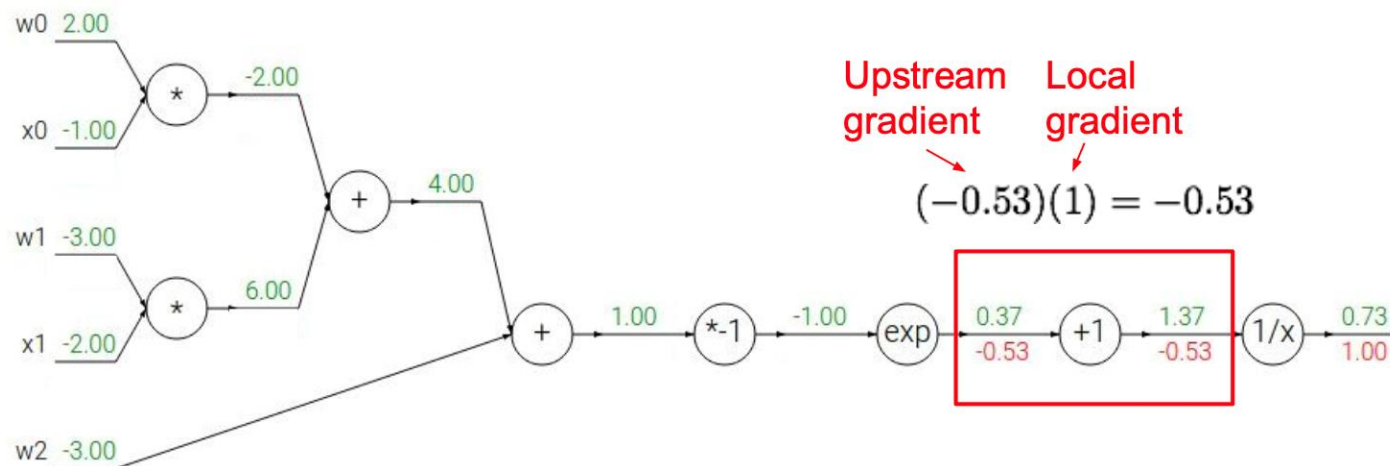
$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$



$f(x) = e^x$	$\rightarrow$	$\frac{df}{dx} = e^x$		$f(x) = \frac{1}{x}$	$\rightarrow$	$\frac{df}{dx} = -1/x^2$
$f_a(x) = ax$	$\rightarrow$	$\frac{df}{dx} = a$		$f_c(x) = c + x$	$\rightarrow$	$\frac{df}{dx} = 1$

Another example:

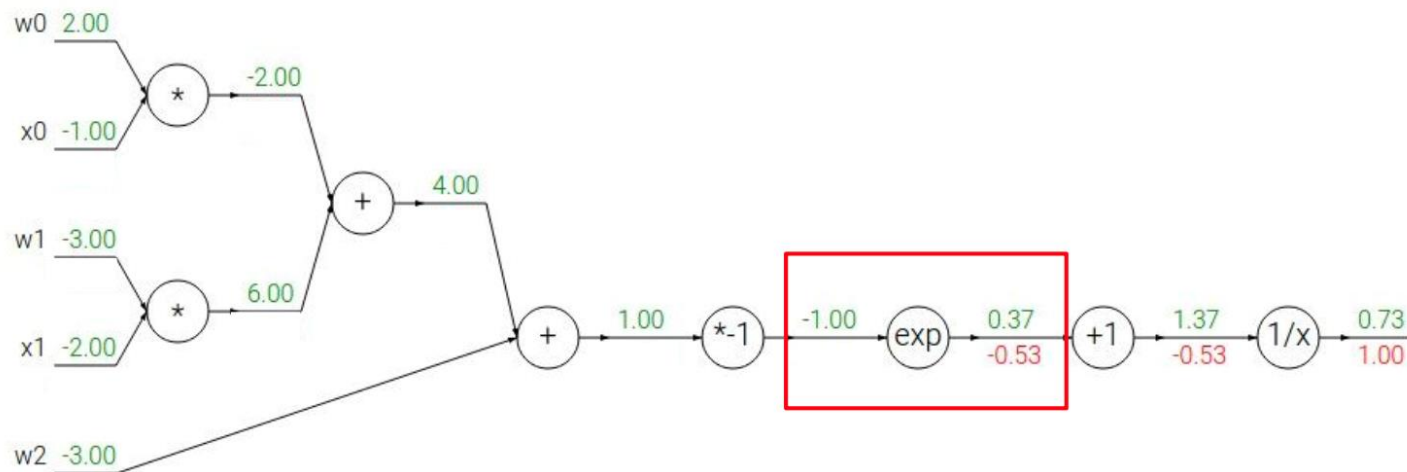
$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$



$f(x) = e^x$	$\rightarrow$	$\frac{df}{dx} = e^x$		$f(x) = \frac{1}{x}$	$\rightarrow$	$\frac{df}{dx} = -1/x^2$
$f_a(x) = ax$	$\rightarrow$	$\frac{df}{dx} = a$		$f_c(x) = c + x$	$\rightarrow$	$\frac{df}{dx} = 1$

Another example:

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$



$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

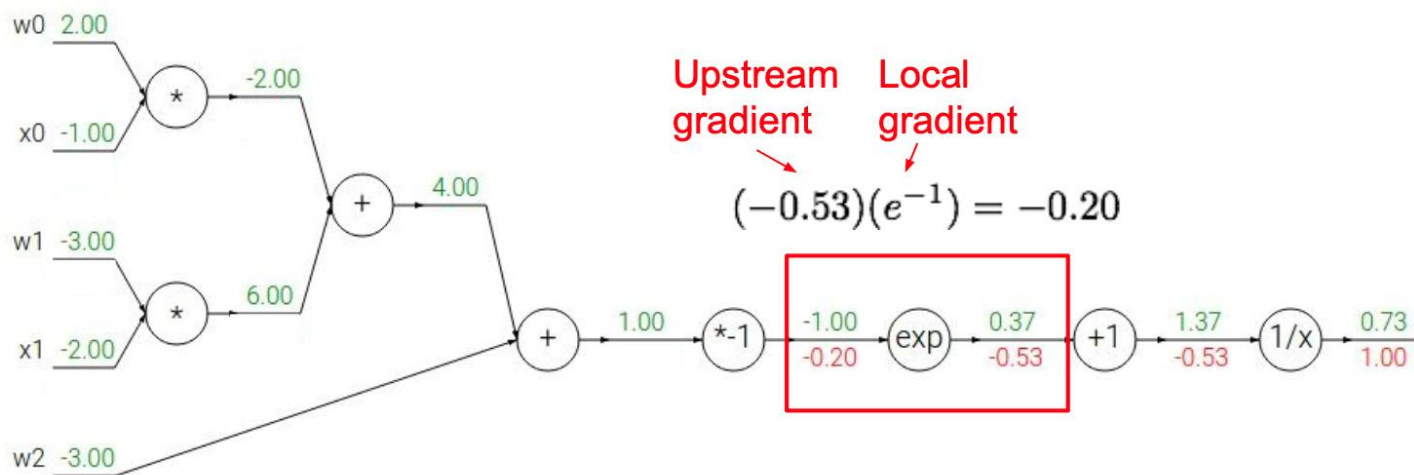
$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Another example:

$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$



$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x}$$

$$f_c(x) = c + x$$

$\rightarrow$

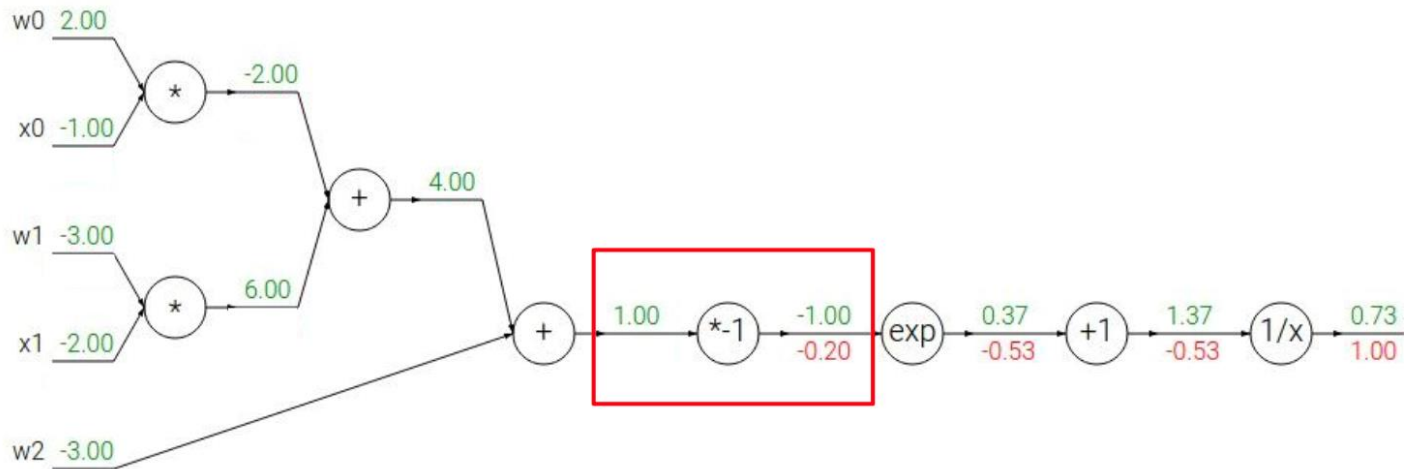
$$\frac{df}{dx} = -1/x^2$$

$\rightarrow$

$$\frac{df}{dx} = 1$$

Another example:

$$f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$



$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

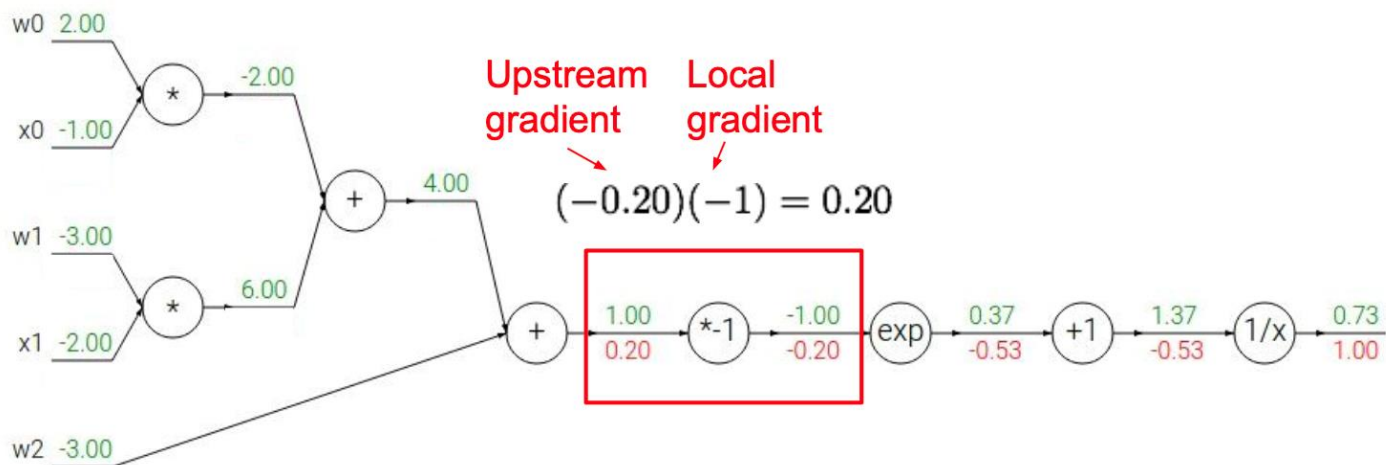
$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Another example:

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$



$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

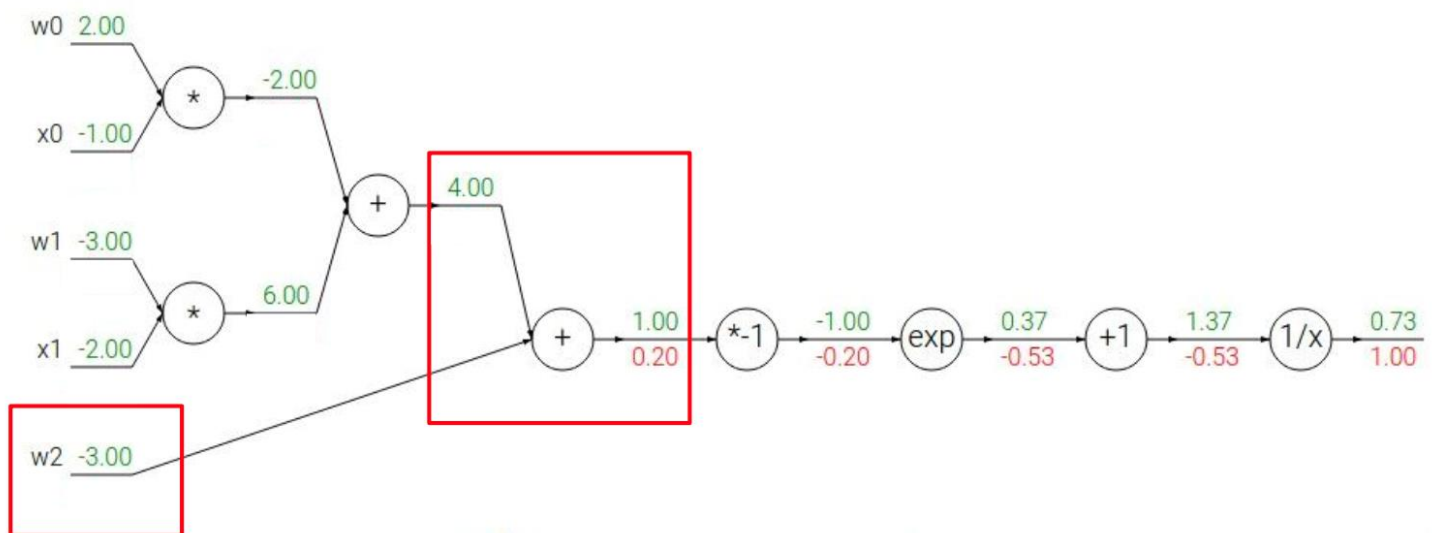
$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$



Another example:

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2x_2)}}$$



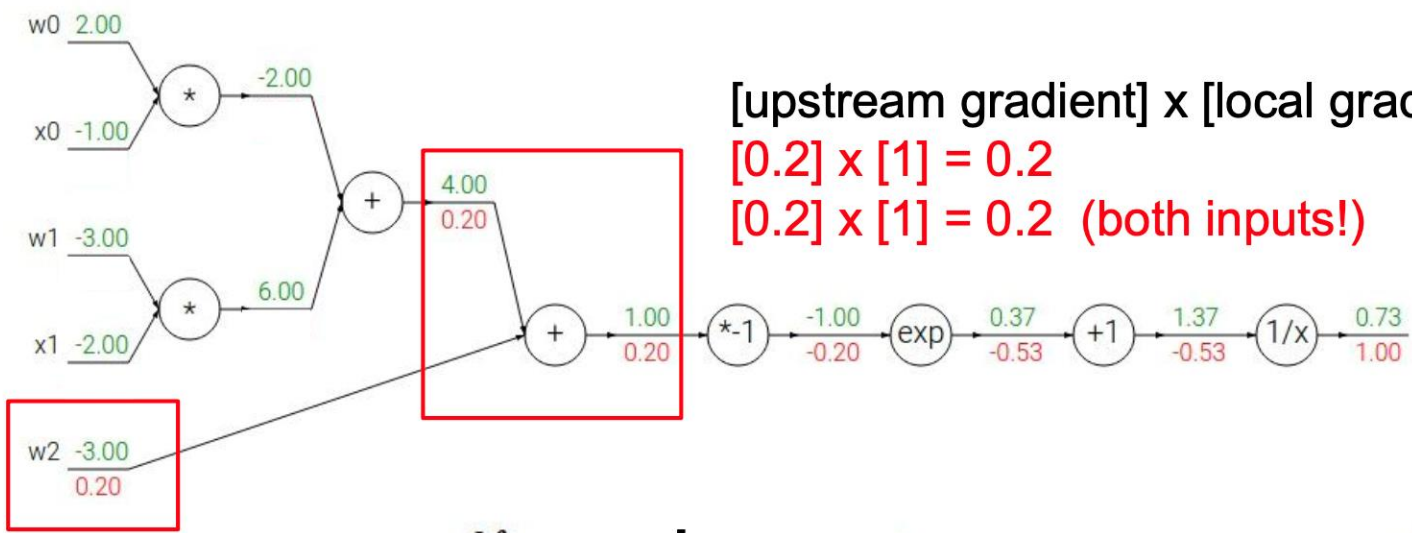
$f(x) = e^x$	$\rightarrow$	$\frac{df}{dx} = e^x$		$f(x) = \frac{1}{x}$	$\rightarrow$	$\frac{df}{dx} = -1/x^2$
$f_a(x) = ax$	$\rightarrow$	$\frac{df}{dx} = a$		$f_c(x) = c + x$	$\rightarrow$	$\frac{df}{dx} = 1$





Another example:

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$



[upstream gradient] x [local gradient]
[0.2] x [1] = 0.2
[0.2] x [1] = 0.2 (both inputs!)

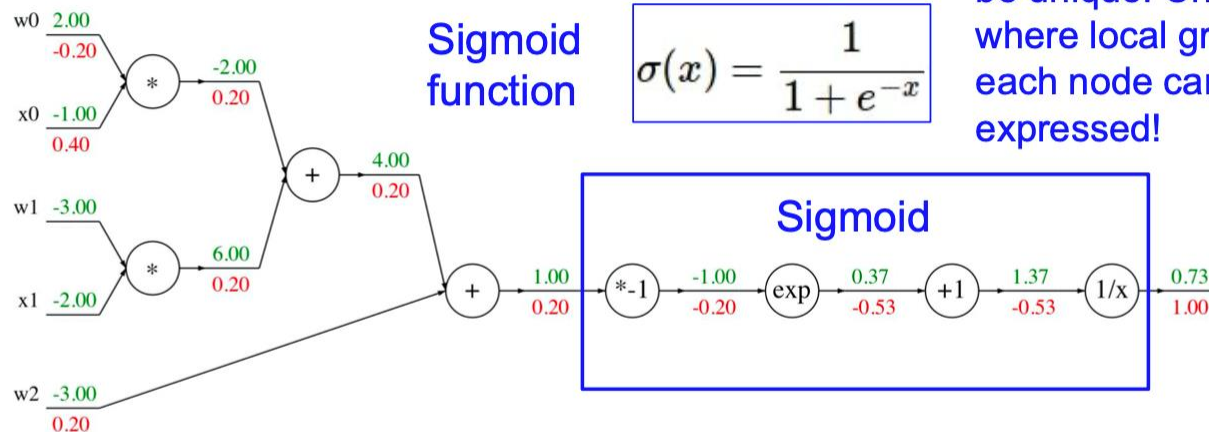
$f(x) = e^x$	$\rightarrow$	$\frac{df}{dx} = e^x$		$f(x) = \frac{1}{x}$	$\rightarrow$	$\frac{df}{dx} = -1/x^2$
$f_a(x) = ax$	$\rightarrow$	$\frac{df}{dx} = a$		$f_c(x) = c + x$	$\rightarrow$	$\frac{df}{dx} = 1$



Another example:

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$

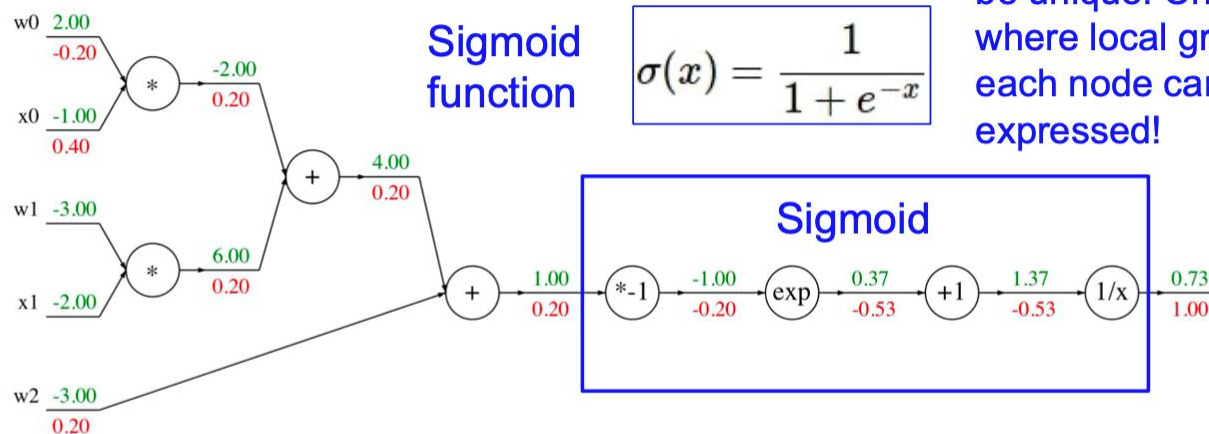
Computational graph representation may not be unique. Choose one where local gradients at each node can be easily expressed!



Another example:

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$

Computational graph representation may not be unique. Choose one where local gradients at each node can be easily expressed!



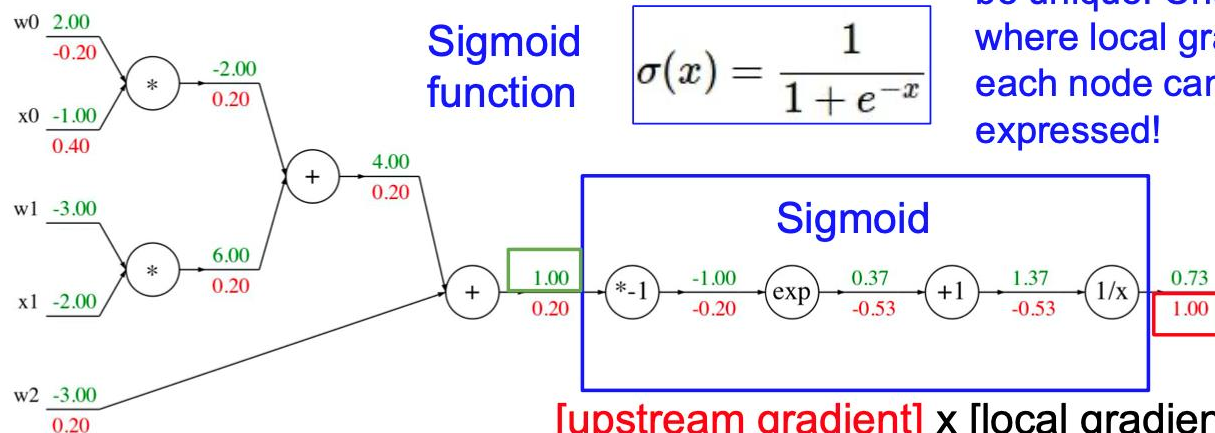
Sigmoid local gradient:

$$\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1 + e^{-x})^2} = \left(\frac{1 + e^{-x} - 1}{1 + e^{-x}} \right) \left(\frac{1}{1 + e^{-x}} \right) = (1 - \sigma(x)) \sigma(x)$$

Another example:

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2x_2)}}$$

Computational graph representation may not be unique. Choose one where local gradients at each node can be easily expressed!



Sigmoid function

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Sigmoid

[upstream gradient] x [local gradient]
 $[1.00] \times [(1 - 1/(1+e^1)) (1/(1+e^1))] = 0.2$

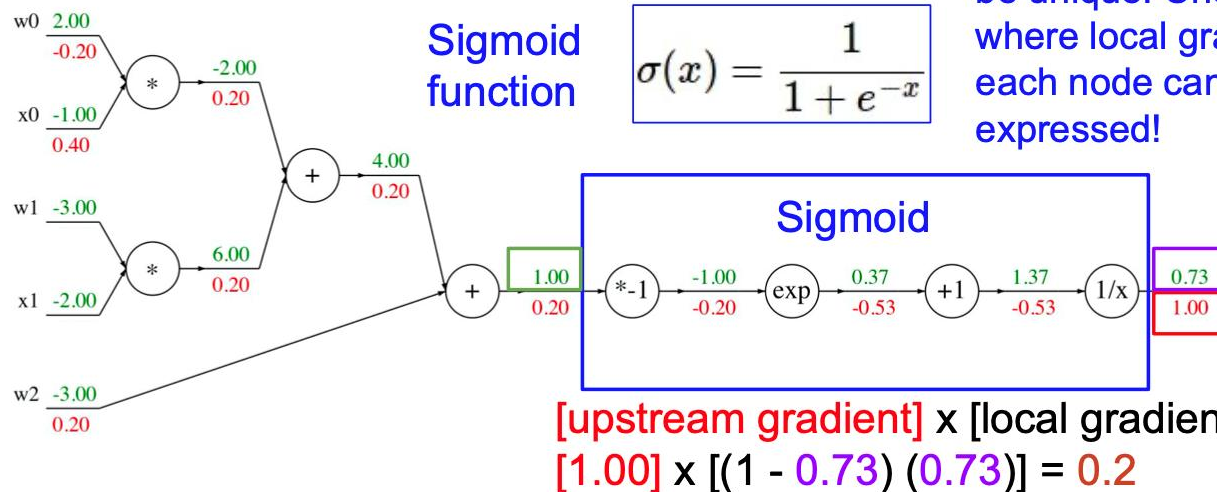
Sigmoid local gradient:

$$\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1 + e^{-x})^2} = \left(\frac{1 + e^{-x} - 1}{1 + e^{-x}} \right) \left(\frac{1}{1 + e^{-x}} \right) = (1 - \sigma(x)) \sigma(x)$$

Another example:

$$f(w, x) = \frac{1}{1 + e^{-(w_0x_0 + w_1x_1 + w_2)}}$$

Computational graph representation may not be unique. Choose one where local gradients at each node can be easily expressed!

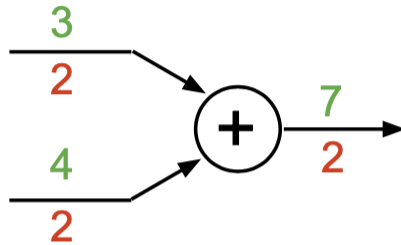


Sigmoid local gradient:

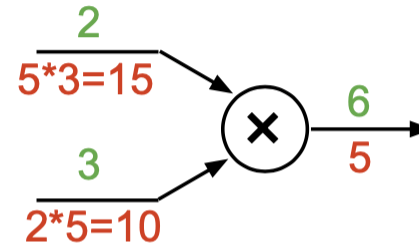
$$\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1 + e^{-x})^2} = \left(\frac{1 + e^{-x} - 1}{1 + e^{-x}} \right) \left(\frac{1}{1 + e^{-x}} \right) = (1 - \sigma(x)) \sigma(x)$$

Patterns in gradient flow

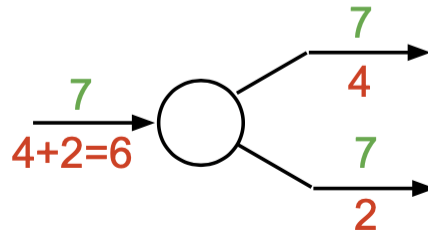
add gate: gradient distributor



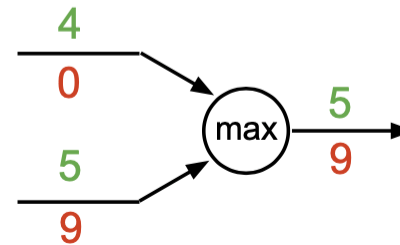
mul gate: “swap multiplier”



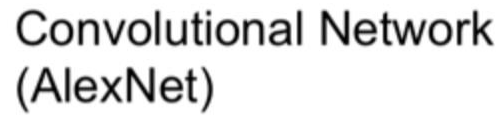
copy gate: gradient adder



max gate: gradient router



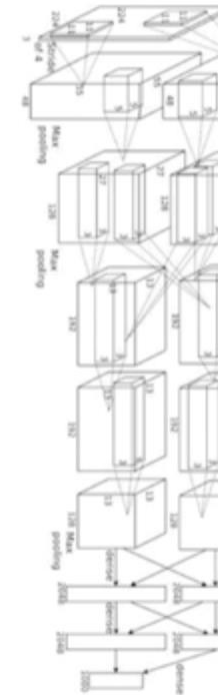
Can be applied to arbitrarily complex deep networks



input image

weights

loss





Overall Steps of BP



After choosing the weights of the network randomly, the backpropagation algorithm is used to compute the necessary corrections. The algorithm can be decomposed in the following four steps:

- i) Feed-forward computation
- ii) Backpropagation to the output layer
- iii) Backpropagation to the hidden layer
- iv) Weight updates

The algorithm is stopped when the value of the error function has become sufficiently small.





Multilayer Neural Networks

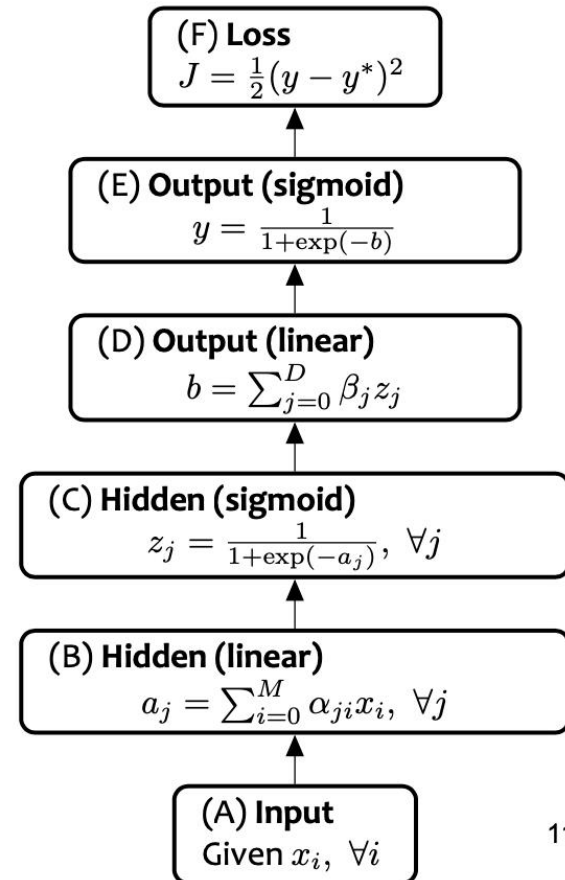
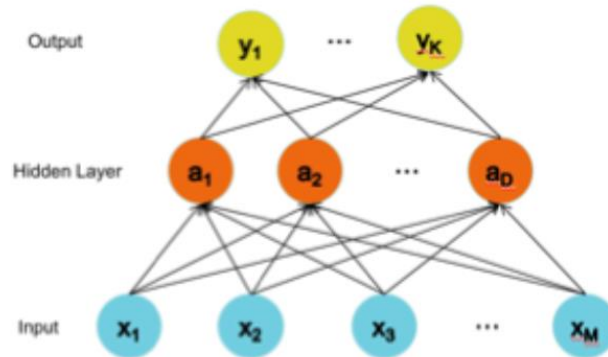
❖ Network Design



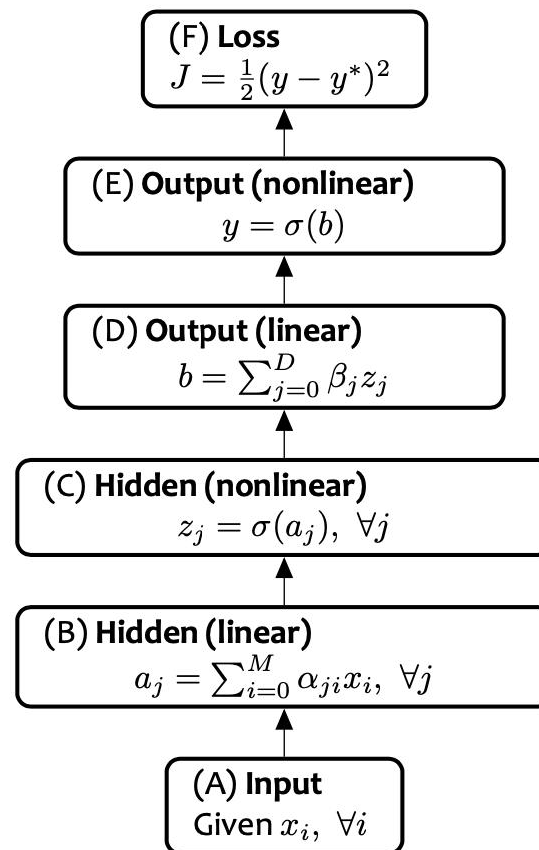
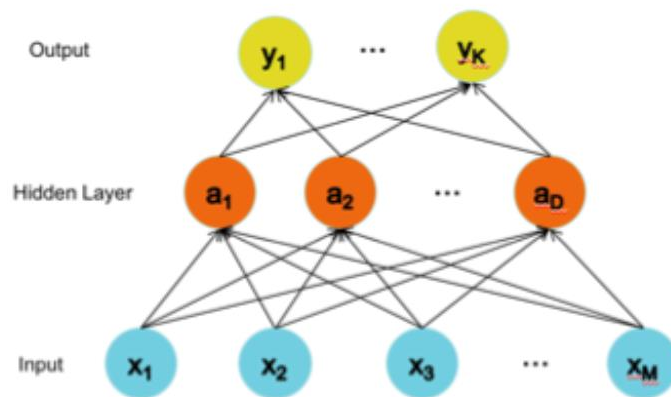
- ❖ **Even for a basic Multilayer Neural Network, there are many design decisions to make:**
 - 1) Number of hidden layers (depth of the network)**
 - 2) Number of neurons per hidden layer (width)**
 - 3) Type of activation functions**
 - 4) Form of objective functions**



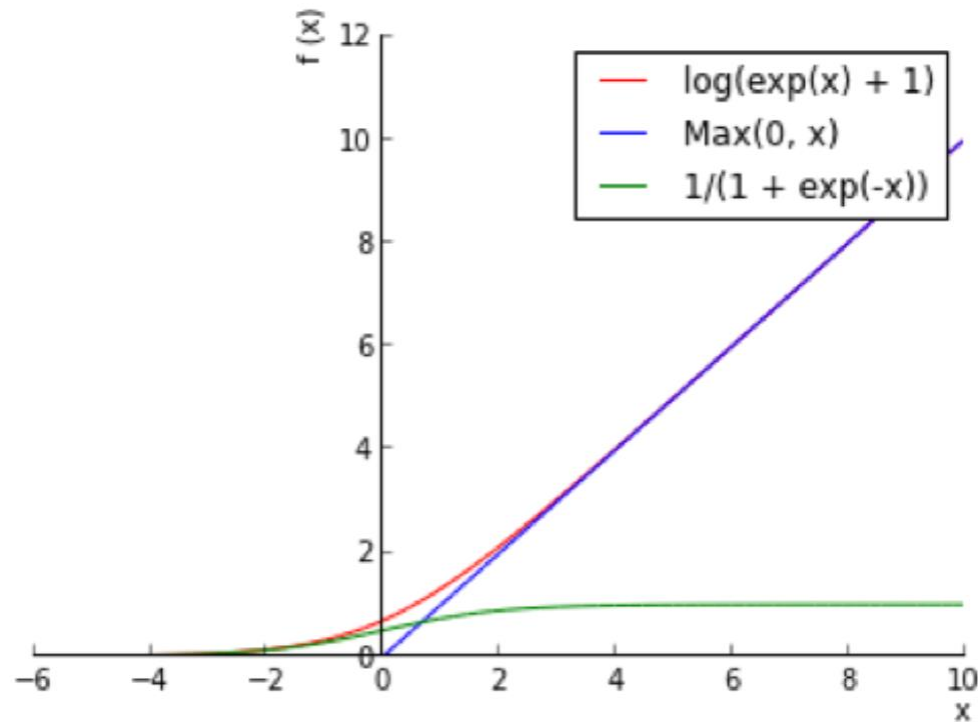
Example: Neural Network with
sigmoid activation functions



Neural Network with arbitrary
nonlinear activation functions



ReLU often used in computer vision tasks

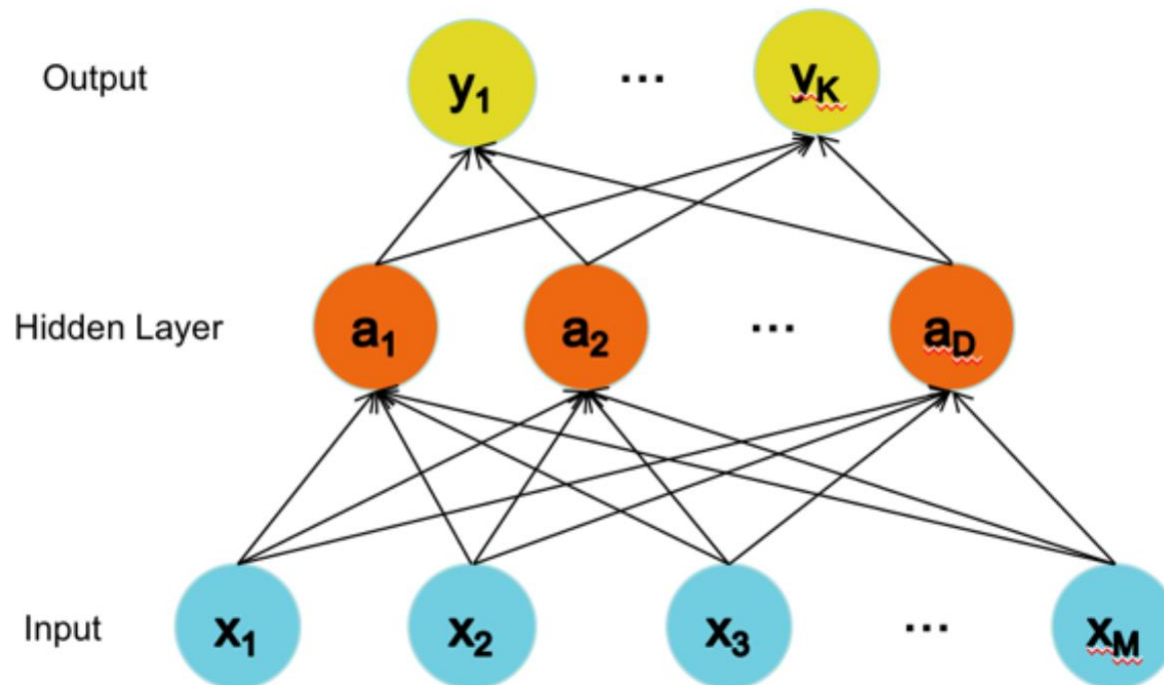


Loss Functions for NN

- Regression:
 - Use the same objective as Linear Regression
 - Quadratic loss (i.e. mean squared error)
- Classification:
 - Use the same objective as Logistic Regression
 - Cross-entropy (i.e. negative log likelihood)
 - This requires probabilities, so we add an additional “softmax” layer at the end of our network

	Forward	Backward
Quadratic	$J = \frac{1}{2}(y - y^*)^2$	$\frac{dJ}{dy} = y - y^*$
Cross Entropy	$J = y^* \log(y) + (1 - y^*) \log(1 - y)$	$\frac{dJ}{dy} = y^* \frac{1}{y} + (1 - y^*) \frac{1}{y - 1}$

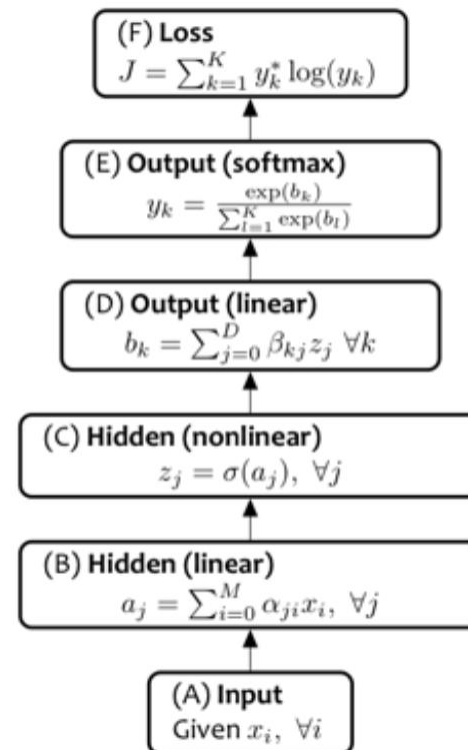
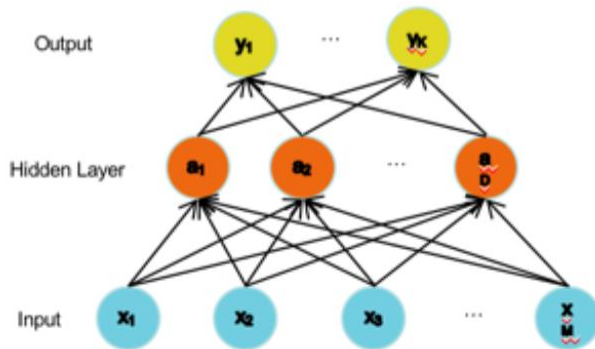
Multi-class Output



Multi-class Output

Softmax:

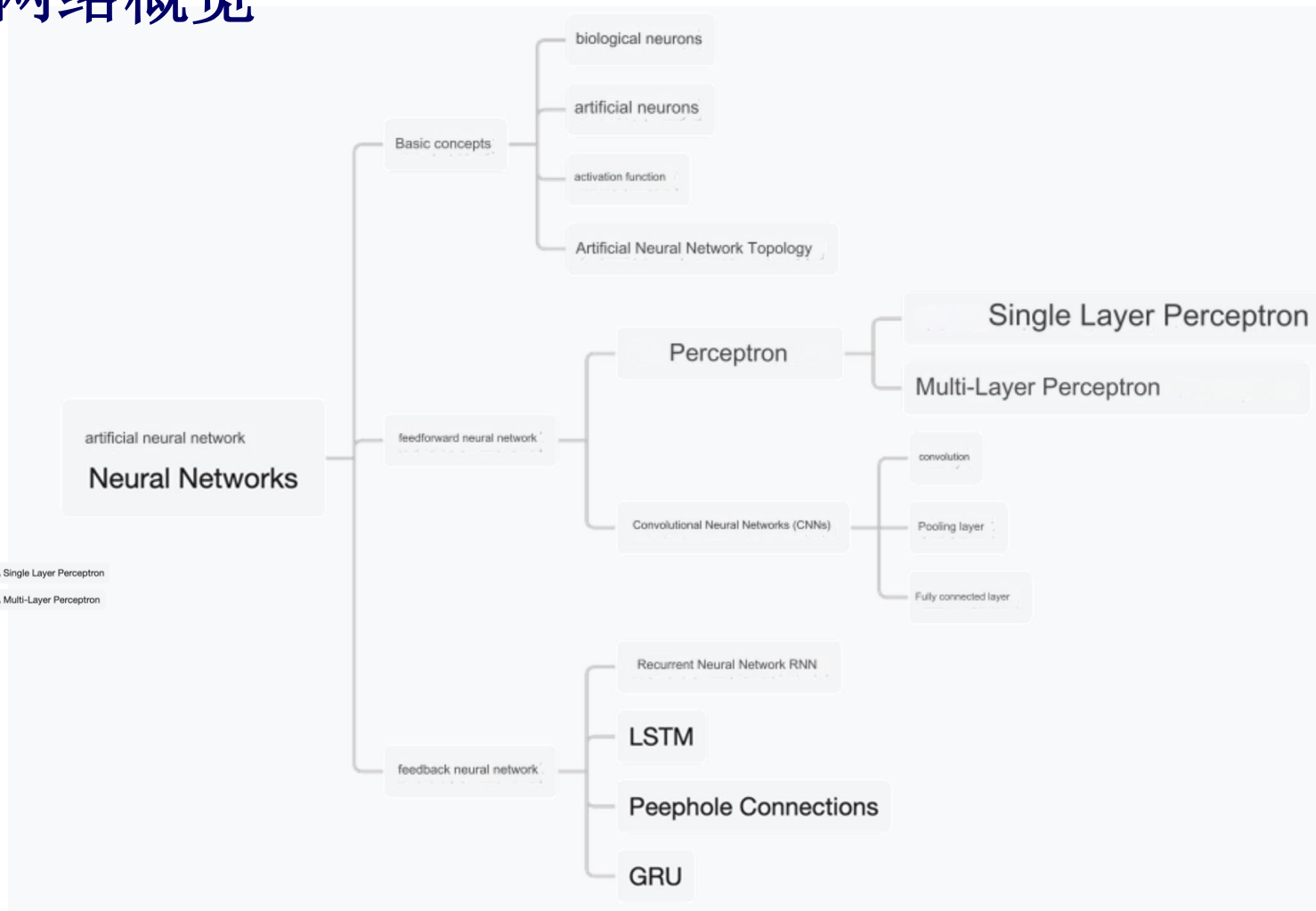
$$y_k = \frac{\exp(b_k)}{\sum_{l=1}^K \exp(b_l)}$$



- ❖ **Chapter 6 Deep feedforward networks, Deep Learning,**
[https://www.deeplearningbook.org](https://www.deeplearningbook.org/L)
L

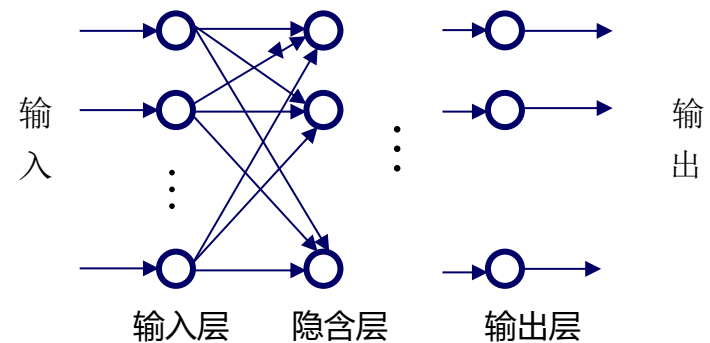


❖ 人工神经网络概览



Feedforward Neural Network

- ❖ Feedforward Neural Network (FNN) is a basic artificial neural network structure, in which information flows in one direction, from the input layer through one or more hidden layers to the output layer. There is no feedback between neurons.

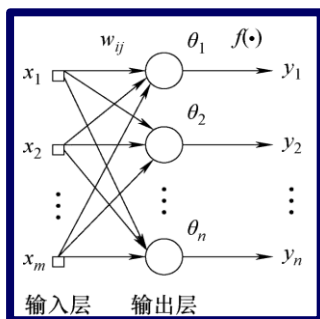


Feedforward Neural Network

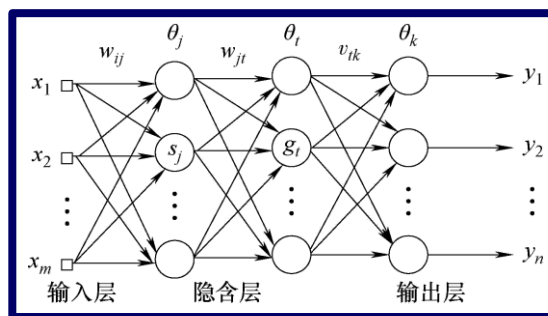
Feedforward Neural Network

❖ Perceptron

proposed the Perceptron model for pattern classification. Stacking single-layer perceptrons can produce a multi-layer perceptron network with stronger learning ability. ◦



单层感知器



多层感知器

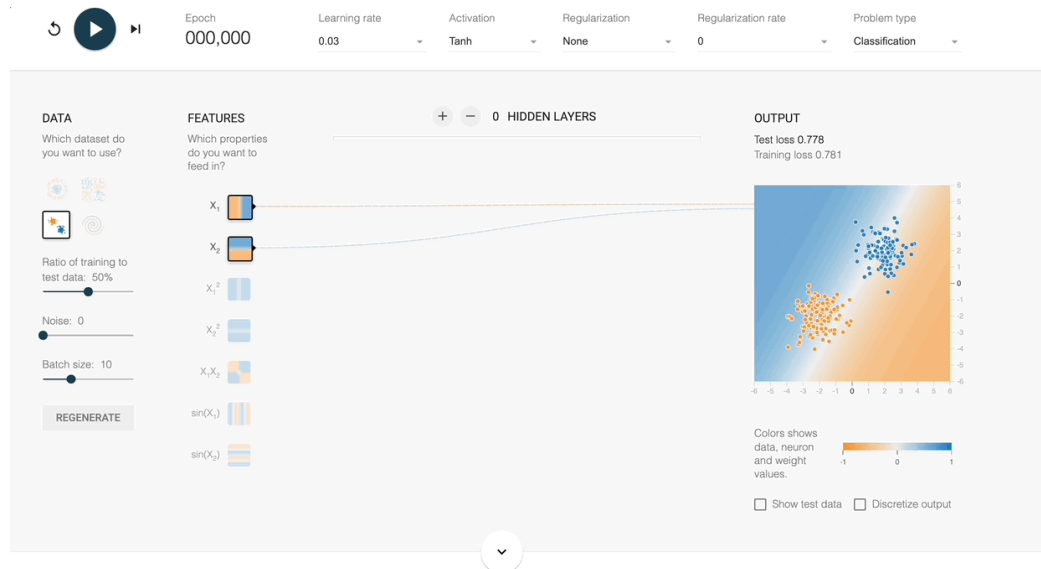


Feedforward Neural Network

❖ Perceptron

- **Single layer**

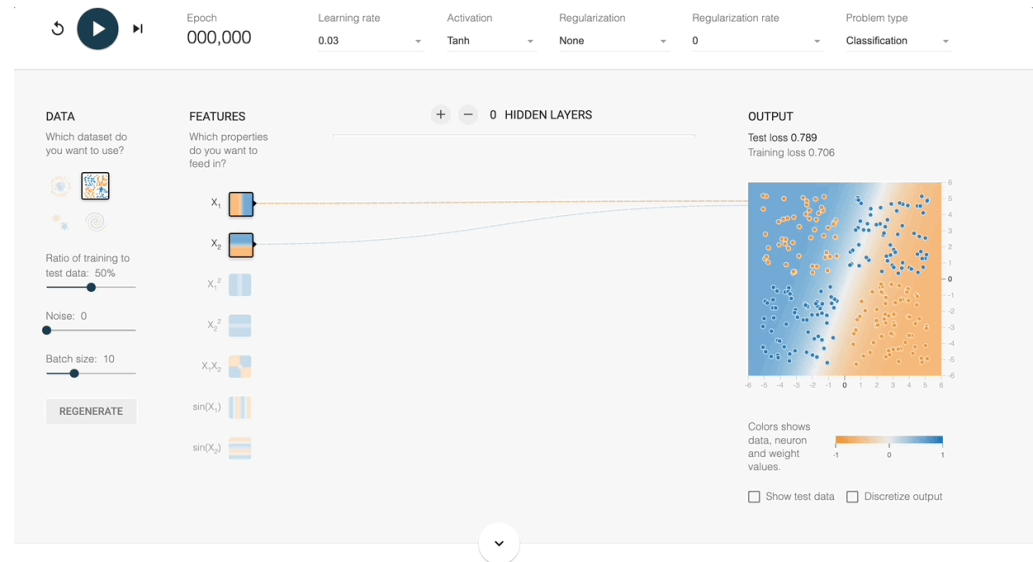
- 如右图所示，单层感知器可以迅速找到两类数据的线性边界



Feedforward Neural Network

❖ Perceptron

- Single layer cannot solve ...



- 但单层感知器不能解决“异或”问题。因为其分类边界是线性的。
- 如右图所示，单层感知器无法找到两类数据的分类边界。



Feedforward Neural Network

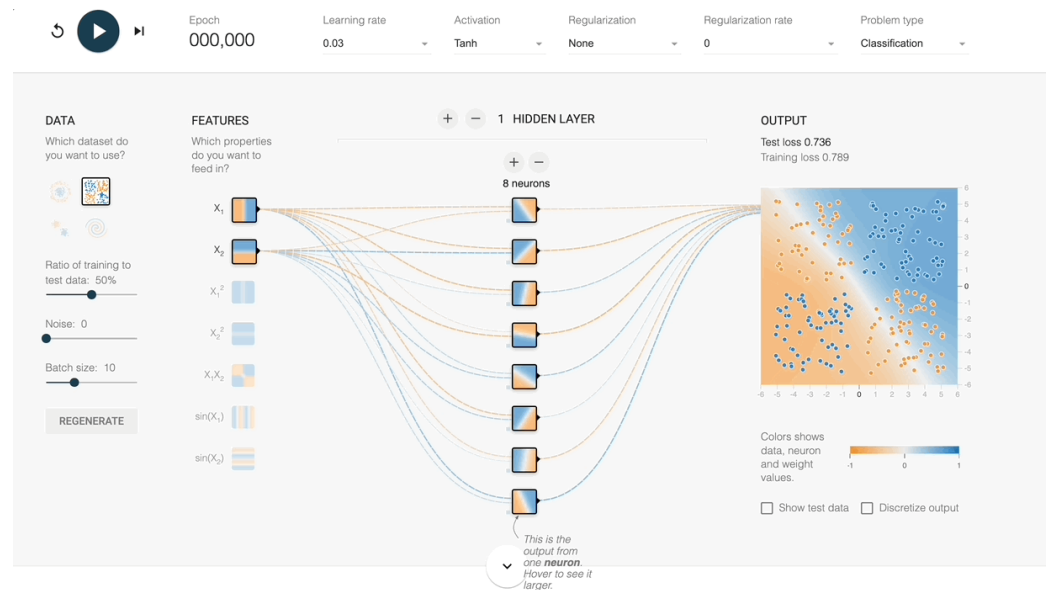
❖ Perceptron

• Multi-layer

The classification boundary of the multi-layer perceptron is nonlinear and can solve the "XOR" problem.

As shown in the figure on the right, the two-layer perceptron quickly learns to find the classification boundary of the XOR data.

- 多层感知器分类边界是非线性的，能解决“异或”问题。
- 如右图所示，两层感知器迅速学习找到了异或数据的分类边界。



Demo website:
<https://playground.tensorflow.org/>

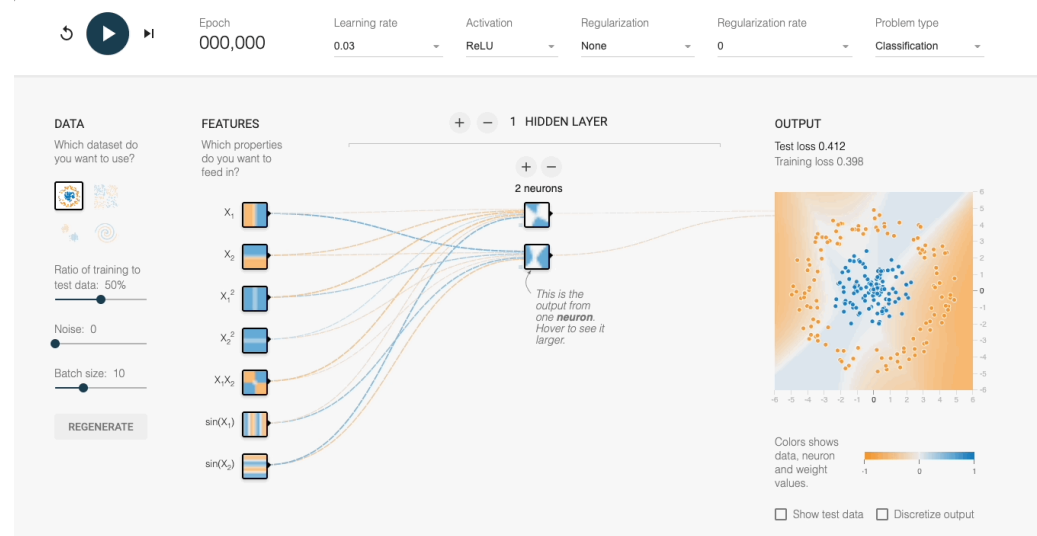
Feedforward Neural Network

❖ Perceptron

In addition to the XOR problem, the two-layer perceptron can also solve data patterns of similar complexity.

As shown in the figure on the right, the two-layer perceptron quickly learns to find the classification boundaries of the data.

- 除了异或问题，两层感知机还能解决类似复杂程度的数据模式。
- 如右图所示，两层感知器迅速学习找到数据的分类边界

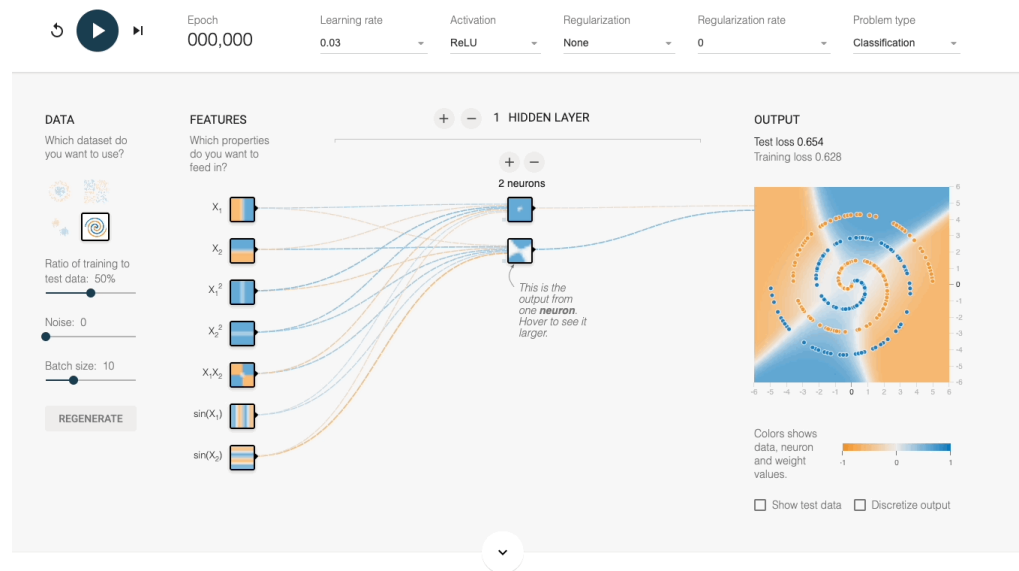


Feedforward Neural Network

❖ Perceptron

But as the complexity of the data increases, more layers of perceptrons are needed to solve it.

As shown in the figure on the right, the learning of two layers of perceptrons stagnates.

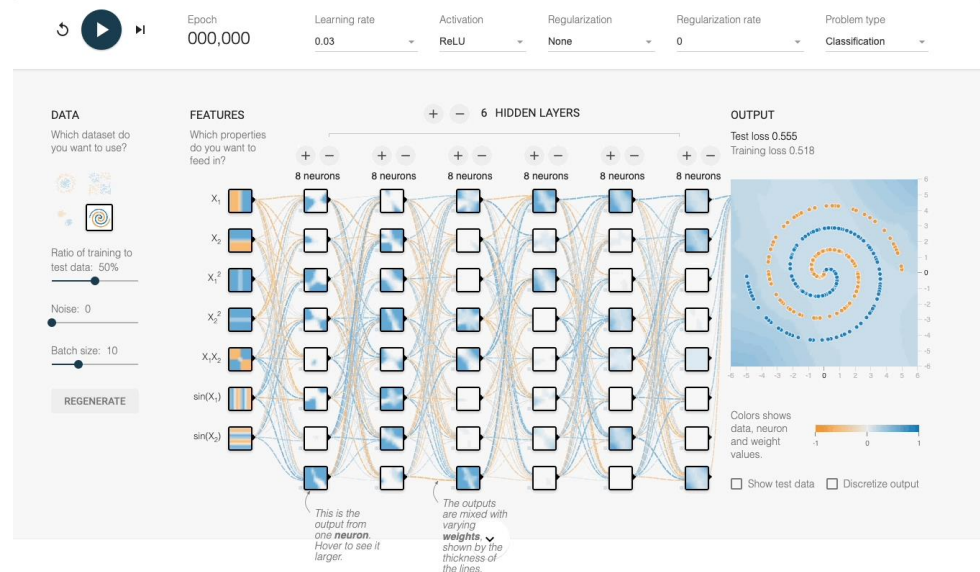


- 但随着数据复杂程度的提高，需要更多层的感知器才能解决。
- 如右图所示，两层感知器的学习陷入停滞。

Feedforward Neural Network

❖ Perceptron

Due to its more powerful nonlinear representation ability, the seven-layer perceptron can gradually learn to classify complex spiral distribution data that the two-layer perceptron cannot learn.

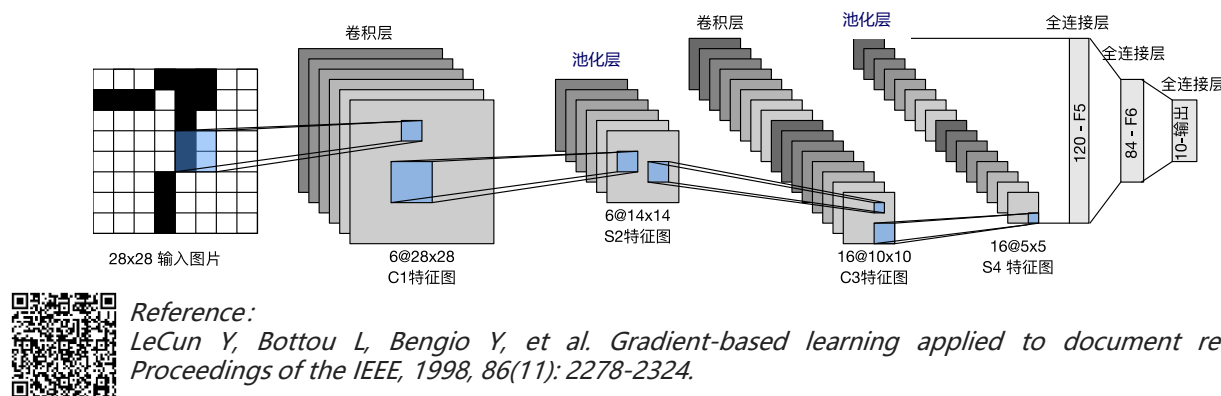


- 由于其更强大的非线性表征能力，七层感知器则能够逐步学会两层感知器学习不了的复杂的螺旋分布数据的分类。

Feedforward Neural Network

❖ CNN

- ❖ The earliest successfully applied convolutional neural network LeNet was proposed by Yann LeCun et al. in 1998.

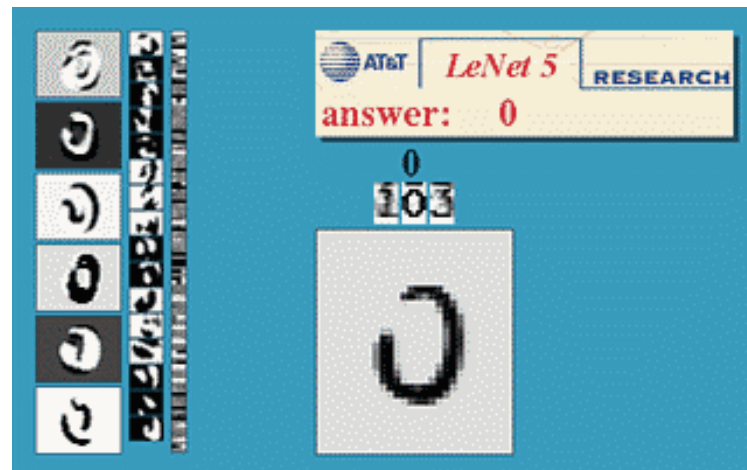


Feedforward Neural Network

❖ 卷积神经网络

❖ LeNet

It is mainly used for handwritten digit recognition tasks and is an important milestone in the field of deep learning and computer vision.



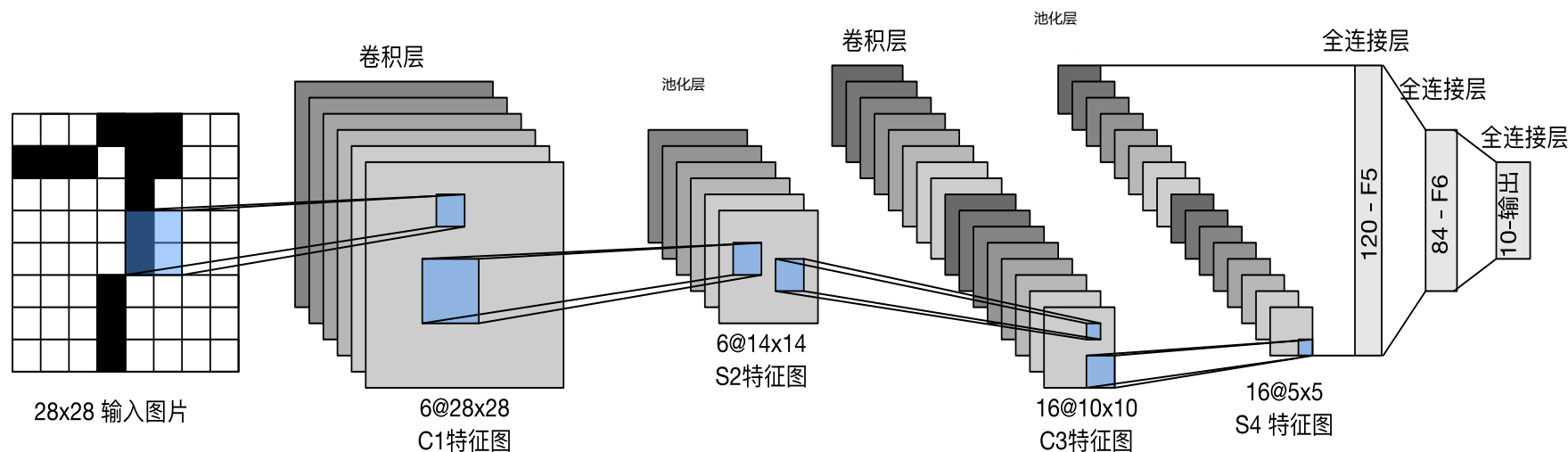
Reference:

LeCun Y, Bottou L, Bengio Y, et al. Gradient-based learning applied to document recognition[J]. *Proceedings of the IEEE*, 1998, 86(11): 2278-2324.

Feedforward Neural Network

❖ CNN

❖ **CNN: 1) Convolutional layer (often followed by an activation function), 2) Pooling layer, 3) Fully connected layer.**



❖ 卷积层

In convolutional neural networks (CNNs), two-dimensional discrete convolutions are often used to extract features from input data (usually images). The convolution operation involves sliding a small matrix (called a convolution kernel or filter) over the input data and calculating the dot product between the convolution kernel and a local area of the input data:

$$(I * K)(x, y) = \sum_i \sum_j I(x + i, y + j) \cdot K(i, j)$$

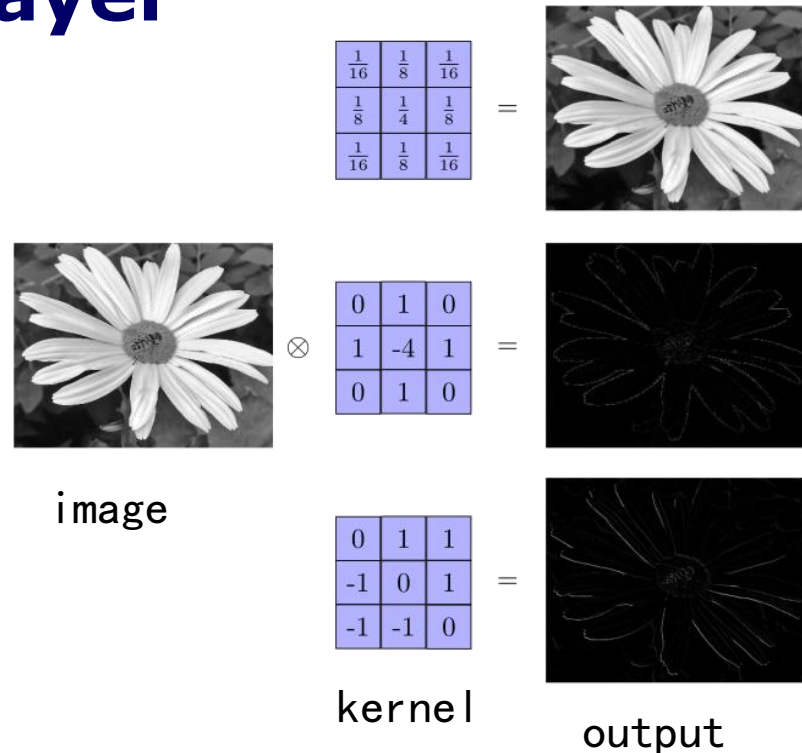
其中：

- $I(x, y)$ 是输入，例如图像
- $K(i, j)$ 是卷积核
- (x, y) 是输出特征图的坐标
- (i, j) 是卷积核的坐标

Feedforward Neural Network

❖ Convolutional layer

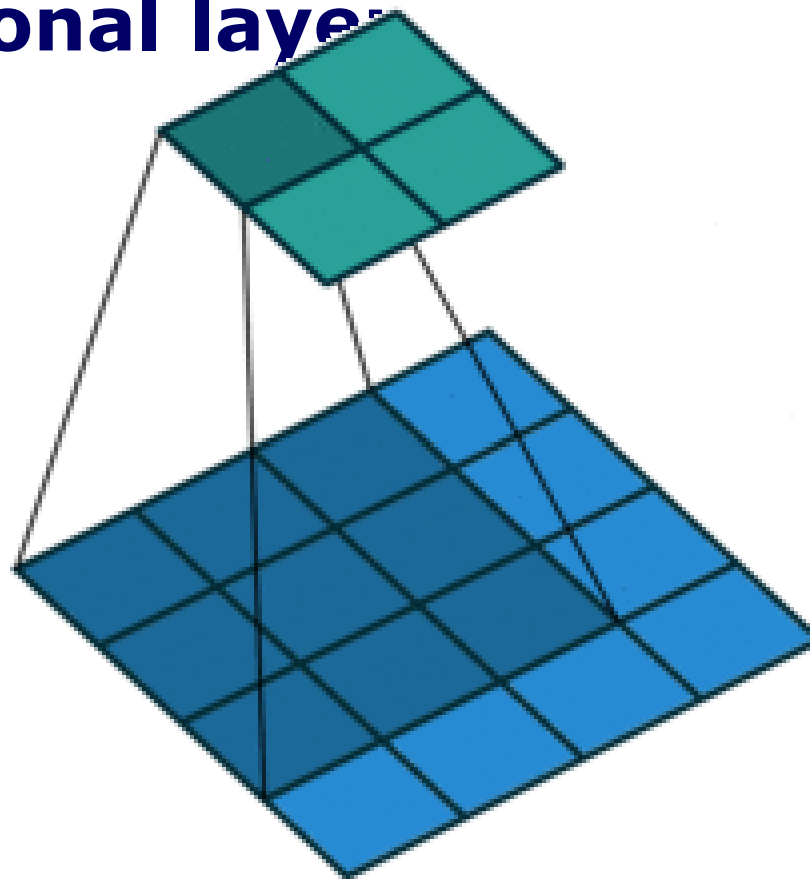
The function of the convolution layer is to extract specific features, such as specific textures or edges. The output image of the convolution kernel is often called a feature map. Depending on the convolution kernel, the extracted features are also different.



- 卷积层的作用是提取特定的特征，例如特定的纹理或者边缘。
- 卷积核的输出图像常被称为特征图（**feature map**）。
- 根据卷积核的不同，提取到的特征也不相同。

Feedforward Neural Network

❖ Convolutional layer

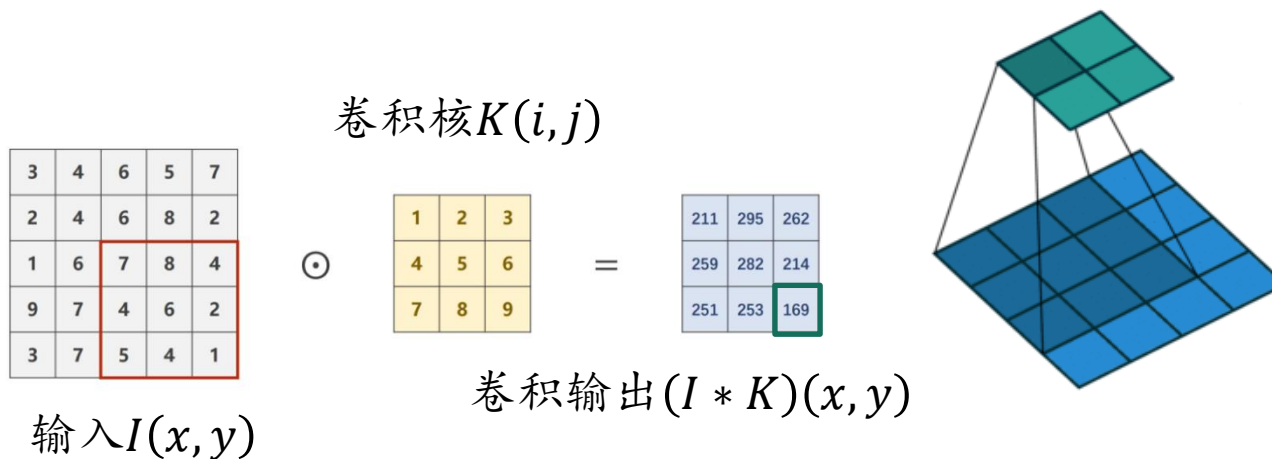


- ❖ 卷积神经网络中，常使用二维离散卷积，用于提取输入数据（通常是图像）的特征。卷积操作即将卷积核在输入数据上滑动，并计算卷积核与输入数据的局部区域之间的点积：

Feedforward Neural Network

❖ Convolutional layer

❖ The calculation



CNN-output size

- Input: $(N, C_{in}, H_{in}, W_{in})$
- Output: $(N, C_{out}, H_{out}, W_{out})$ where

$$H_{out} = \left\lfloor \frac{H_{in} + 2 \times \text{padding}[0] - \text{dilation}[0] \times (\text{kernel_size}[0] - 1) - 1}{\text{stride}[0]} + 1 \right\rfloor$$

$$W_{out} = \left\lfloor \frac{W_{in} + 2 \times \text{padding}[1] - \text{dilation}[1] \times (\text{kernel_size}[1] - 1) - 1}{\text{stride}[1]} + 1 \right\rfloor$$

<https://blog.csdn.net/xnmc2014>



Feedforward Neural Network

❖ The size

- ❖ Assume that the image width is W , the height is H , and the convolution kernel width and height are both k , then:

The output feature map width is $W - k + 1$

The output feature map height is $H - k + 1$

假设图像的宽度为 W ，高度为 H ，卷积核宽和高均为 k ，则：

- 输出的特征图宽度为 $W - k + 1$
- 输出特征图的高度为 $H - k + 1$

$$\begin{array}{|c|c|c|c|c|} \hline 3 & 4 & 6 & 5 & 7 \\ \hline 2 & 4 & 6 & 8 & 2 \\ \hline 1 & 6 & 7 & 8 & 4 \\ \hline 9 & 7 & 4 & 6 & 2 \\ \hline 3 & 7 & 5 & 4 & 1 \\ \hline \end{array} \quad \odot \quad \begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 211 & 295 & 262 \\ \hline 259 & 282 & 214 \\ \hline 251 & 253 & 169 \\ \hline \end{array}$$

$I(x, y)$ $K(i, j)$ $(I * K)(x, y)$

Feedforward Neural Network

❖ The size

It can be seen that with each convolution operation, the image information will continue to be reduced due to convolution, and finally even reduced to 1.

In this regard, padding is often used to solve this problem.

$$\begin{array}{ccccc} 3 & 4 & 6 & 5 & 7 \\ 2 & 4 & 6 & 8 & 2 \\ 1 & 6 & 7 & 8 & 4 \\ 9 & 7 & 4 & 6 & 2 \\ 3 & 7 & 5 & 4 & 1 \end{array} \odot \begin{array}{ccc} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{array} = \begin{array}{ccc} 211 & 295 & 262 \\ 259 & 282 & 214 \\ 251 & 253 & 169 \end{array}$$

$I(x, y)$ $K(i, j)$ $(I * K)(x, y)$

- ❖ 可以看到，随着每次卷积计算操作，图像信息会不断的因卷积而被缩小，最后甚至缩小到**1**。
- ❖ 对此，常采用填充来解决该问题。

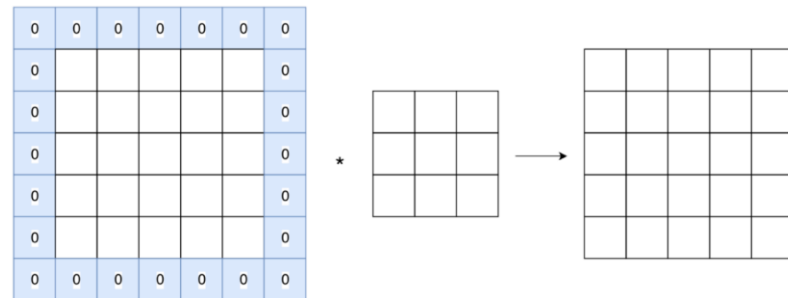
Feedforward Neural Network

❖ padding

- ❖ **Padding** is to fill a certain number of pixels at the edge of the image. During convolution calculation, the convolution kernel can also treat the edge pixels of the input image as the center point.
- ❖ As shown in the right figure, when performing convolution on a 5×5 input, a circle of 1 pixel of 0 is added around it as padding. After convolution calculation, the output is also 5×5 .

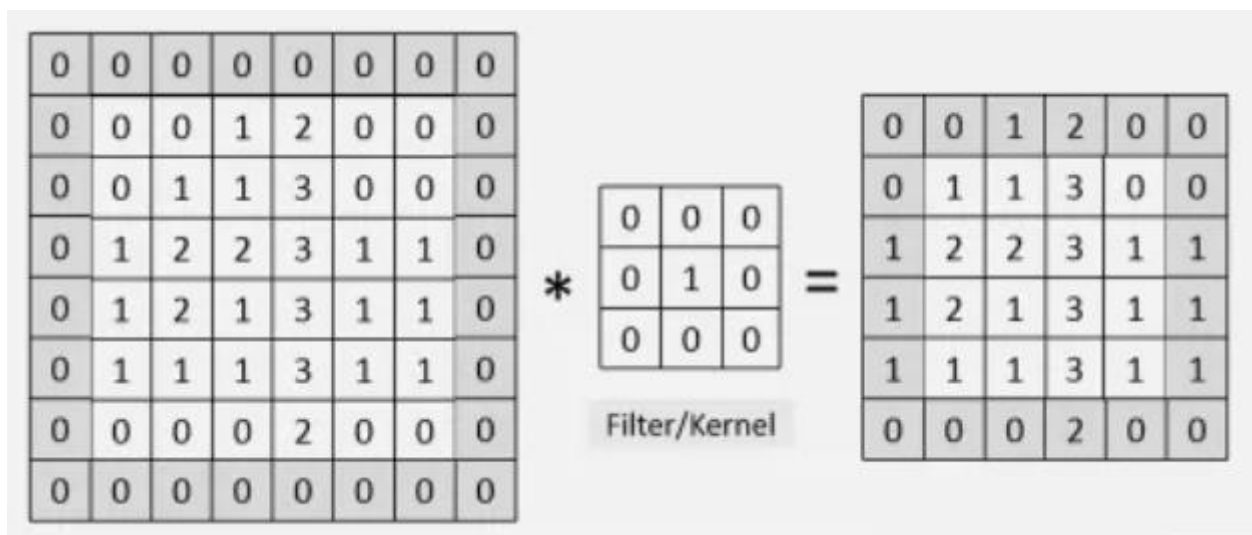
填充（padding）是在图像边缘填充一定的像素点，在卷积计算时，卷积核能把输入图像边缘像素也当做中心点。

右图所示，对 5×5 的输入做卷积时，在周围加入一圈1个像素的0作为填充，卷积计算后，输出也为 5×5 。



padding

❖ Why we need padding



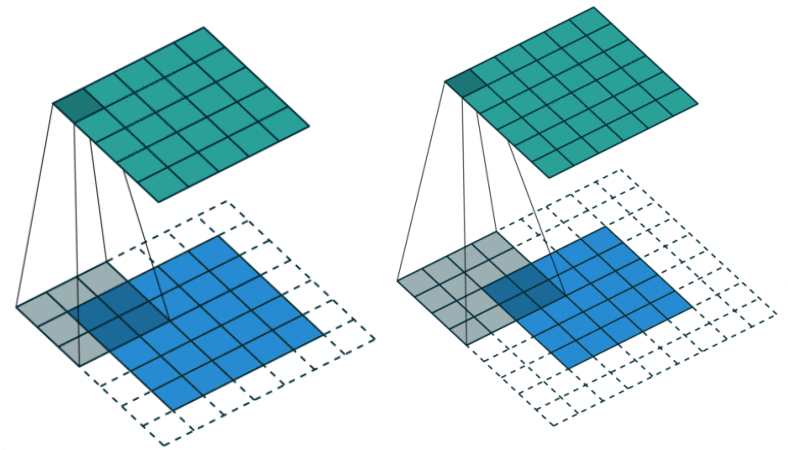
`torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0, dilation=1, groups=1, bias=True, padding_mode='zeros', device=None, dtype=None)`

Feedforward Neural Network

❖ The size (with padding)

- ❖ Assuming the image matrix is $W \times H$ and the convolution kernel is $k \times k$, if P pixels are filled on both sides of the width and height, the size of the output feature map is

$$(W + 2P - k + 1, H + 2P - k + 1)$$



Padding=1

Padding=2

padding=(2,1)

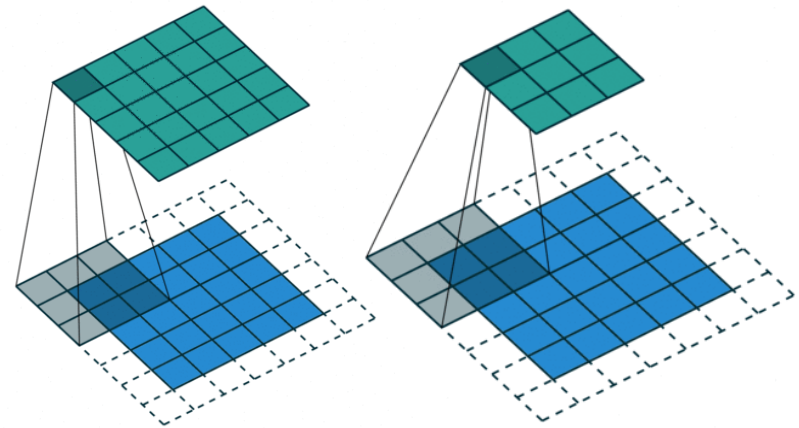
假设图像矩阵为 $W \times H$ ，卷积核为 $k \times k$ ，若在宽和高的单侧均填充 P 个像素点，则输出特征图的大小为

Feedforward Neural Network

❖ stride

卷积计算中还有一个重要参数是步幅（stride），步幅是卷积过程中，卷积核滑动时，两个连续窗口之间的距离。随着步幅增大，卷积层输出尺寸变小。

- ❖ Another important parameter in convolution calculation is stride, which is the distance between two consecutive windows when the convolution kernel slides during the convolution process.
- ❖ As the stride increases, the output size of the convolution layer becomes smaller.

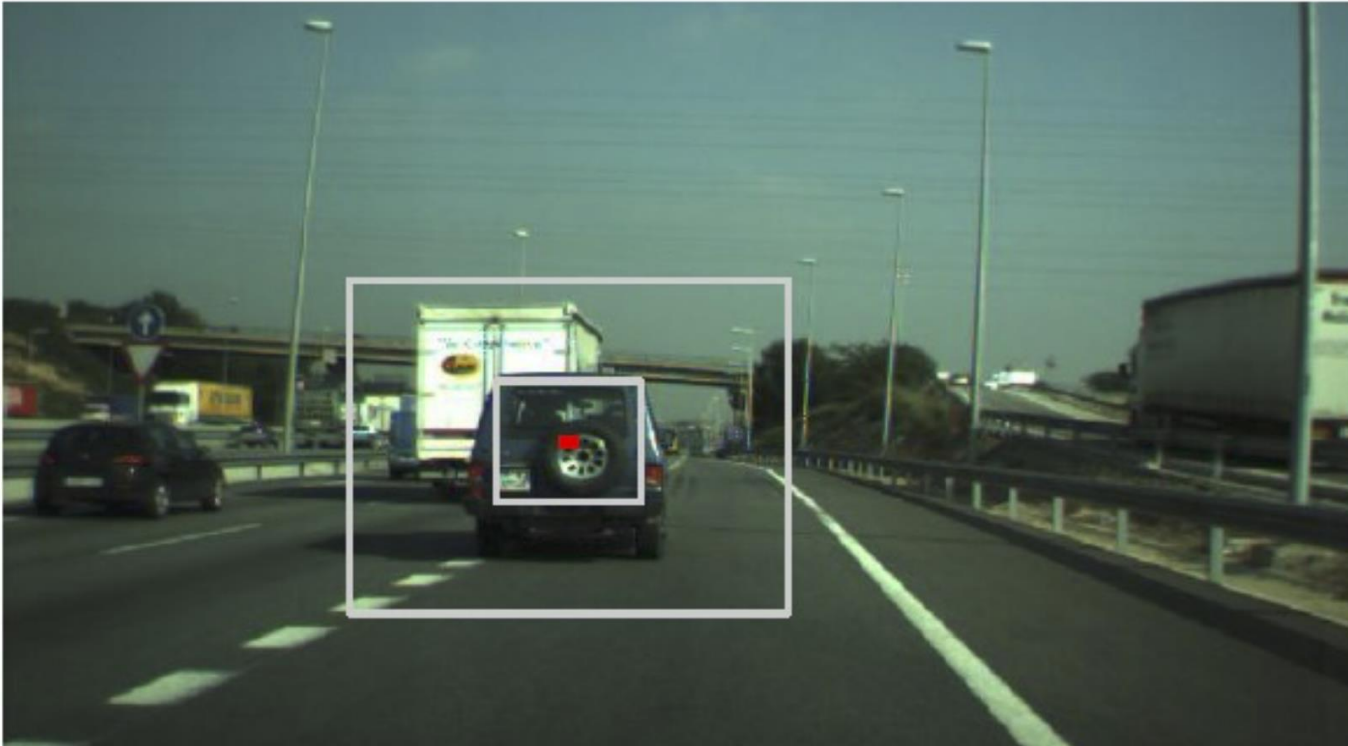


stride=1

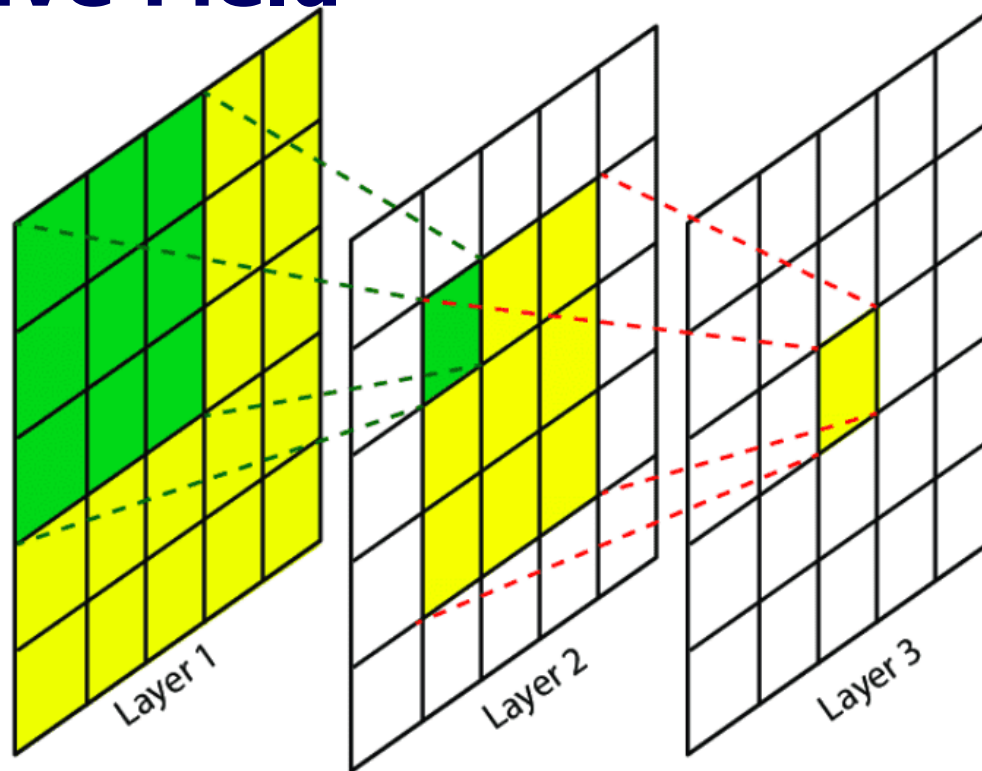
stride=2

Receptive Field

- Is the spatial extent of the connectivity (equivalently this is the filter size).

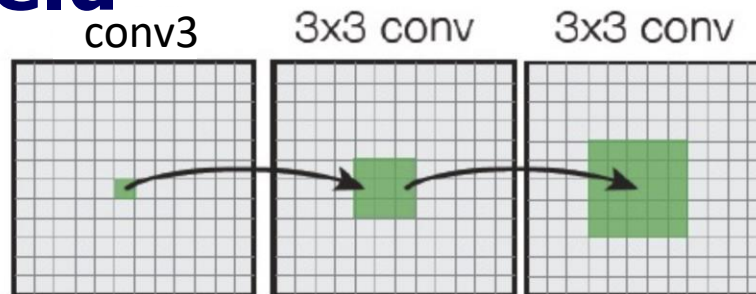


Receptive Field

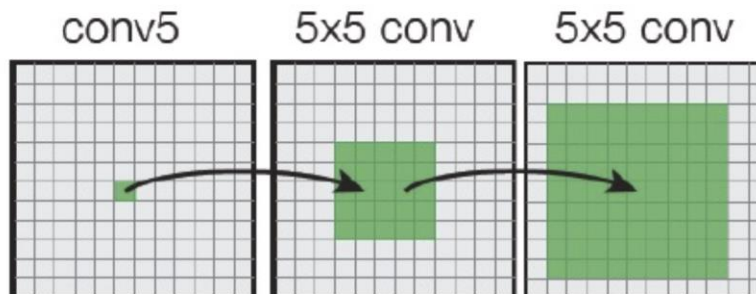


Receptive Field

- Receptive Field



(a) two stacked 3x3 convolution layers



(b) two stacked 5x5 convolution layers

- Receptive field is defined as the image region of the input which influences the feature of each location at the current level(layer).

Feedforward Neural Network

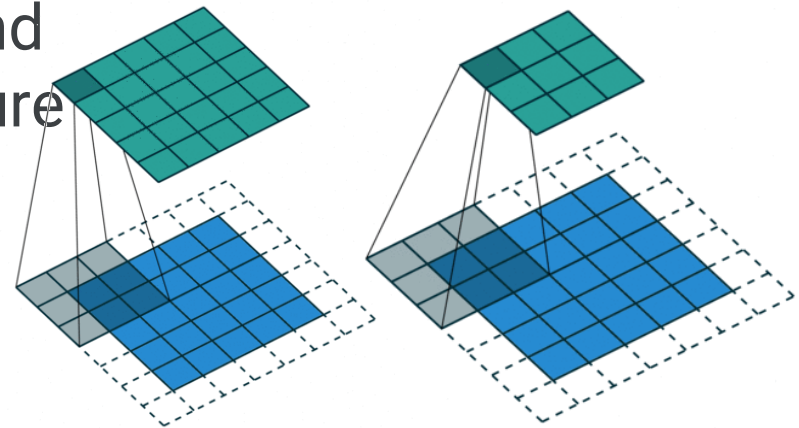
❖ The size (with stride)

- ❖ If the width of the input image is W , the height is H , the convolution kernel size is K , the stride is S , and the padding is P , the output feature map size is calculated as:

$$W' = \frac{W - K + 2P}{S} + 1$$
$$H' = \frac{H - K + 2P}{S} + 1$$

假如输入图像的宽度为 W ，高度为 H ，卷积核尺寸为 K ，步幅为 S ，填充为 P ，计算输出特征图尺寸为：

W' 、 H' 为输出特征图的宽和高。



stride=1

stride=2

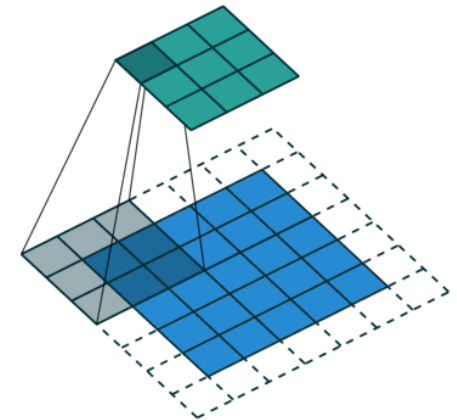
- ❖ $W^{\wedge'}$, $H^{\wedge'}$ are the width and height of the output feature map.

Feedforward Neural Network

❖ The size (with stride)

- ❖ For example, for an input image with a width and height of 5, a convolution kernel size of 3, a stride of 2, and a padding of 1, the width and height of the output feature map are calculated as:

$$\frac{5 - 3 + 2}{2} + 1 = 3$$

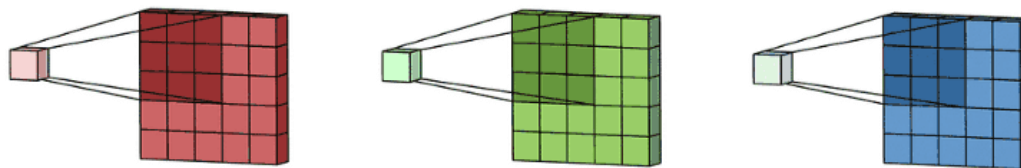


stride=2

Feedforward Neural Network

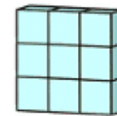
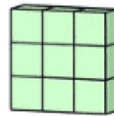
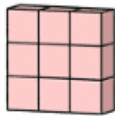
❖ Multi-channel convolution

- ❖ So far, we have assumed that the input is a single channel, but in fact multi-channel convolution is more commonly used. For example, when applying a convolution kernel to an image with an RGB color channel, the convolution kernel can be designed to not have different weights for different channels.



❖ Multi-channel convolution

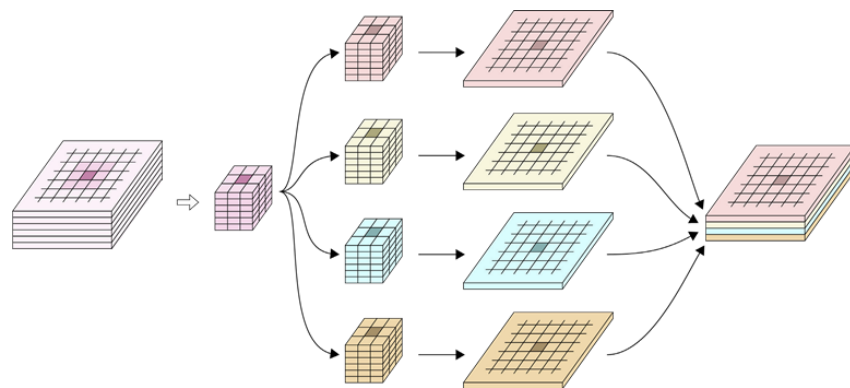
- ❖ Then add the output results of the three channels to get the final result of multi-channel convolution.



Feedforward Neural Network

❖ Multi-channel multi-kernel convolution

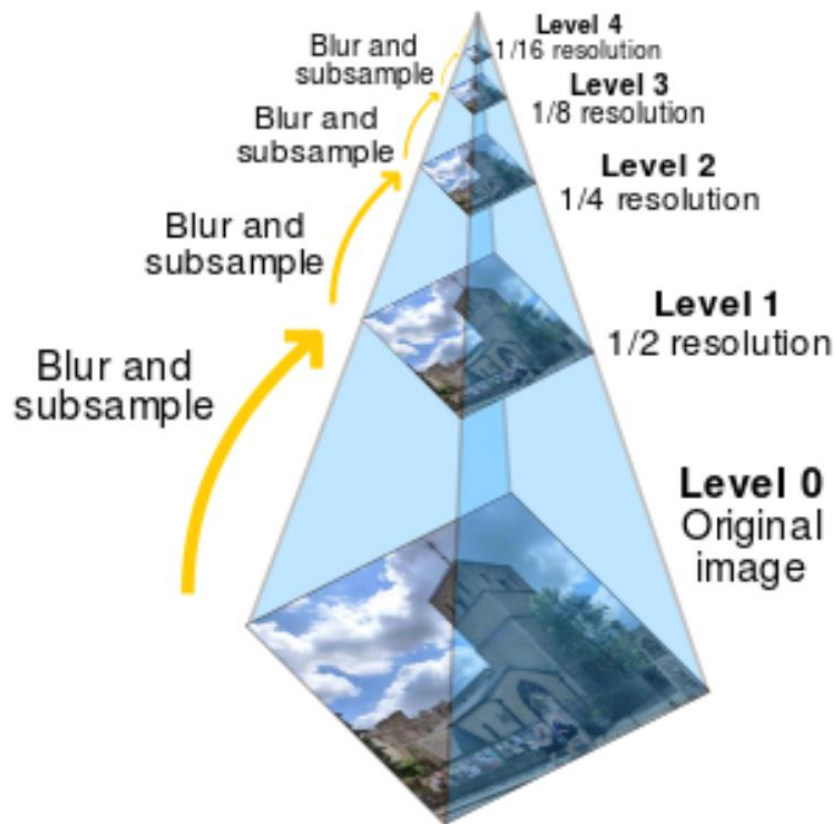
- ❖ A fixed convolution kernel completes the extraction of a certain feature, so when multiple features need to be extracted at the same time, a convolution kernel needs to be created for each channel to form a **multi-channel multi-convolution kernel convolution layer**. At this time, the number of output channels is the number of convolution kernels.



使用多个多通道卷积示意图

Feedforward Neural Network

Pooling is similar to pyramid downsampling



Feedforward Neural Network

❖ Pooling

The features extracted by the convolutional layer are often redundant in space. The purpose of pooling is to simplify the extracted features.

For example, a pooling layer with a stride of 2 and a window size of 2 can downsample the features by a factor of 2.

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

3.5	5.5
11.5	13.5

Average pooling

Max pooling

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

6	8
14	16

Feedforward Neural Network

❖ Pooling

- Pooling often uses statistical information (such as average and maximum values) within a window to characterize the distribution of pixels in the area.
- Depending on the statistical information used, it is called average pooling and maximum pooling.

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

3.5	5.5
11.5	13.5

Average pooling

Max pooling

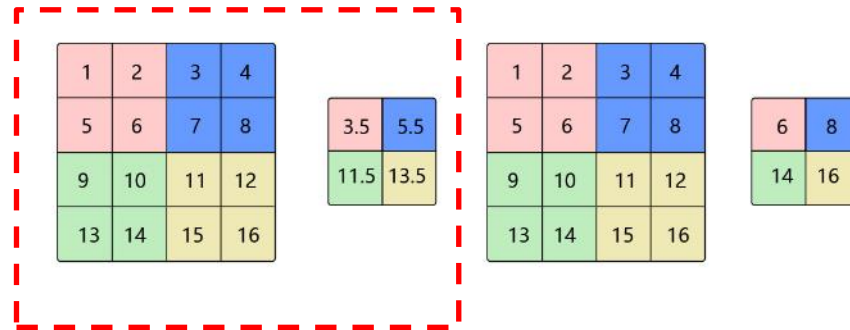
1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

6	8
14	16

Feedforward Neural Network

❖ Pooling

- ❖ Average pooling calculates the mean of all pixels contained in the sub-block of the region and uses the mean as the average pooling result.

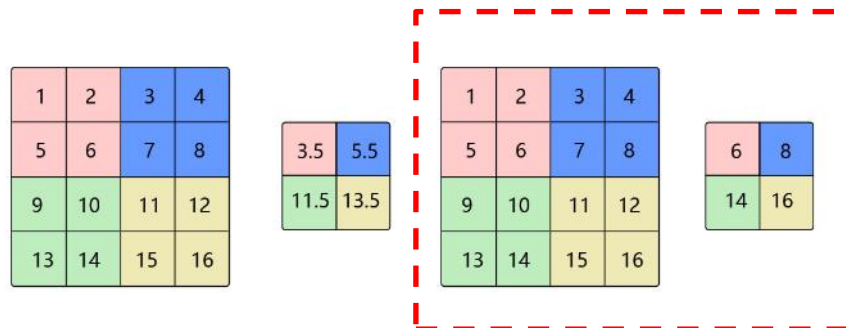


Average pooling

Feedforward Neural Network

❖ Pooling

- ❖ The only difference between max pooling and average pooling is that the pixel with the largest value is selected from a certain area sub-block of the input feature map as the maximum pooling result.



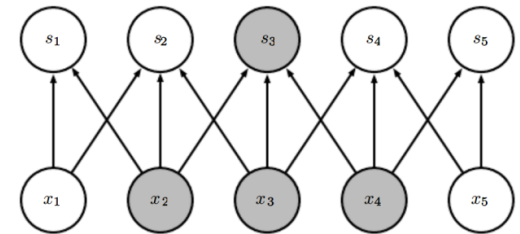
Max pooling

Feedforward Neural Network

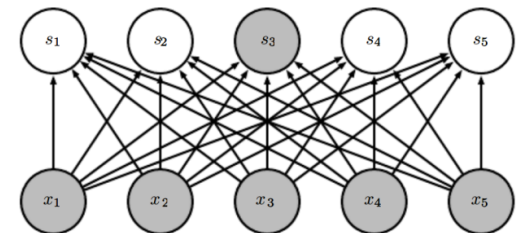
❖ Fully Connected Layer

❖ A fully connected layer is a common layer type in neural networks. In a fully connected layer, every input neuron is connected to every output neuron, hence the name "fully connected".

Conv layer



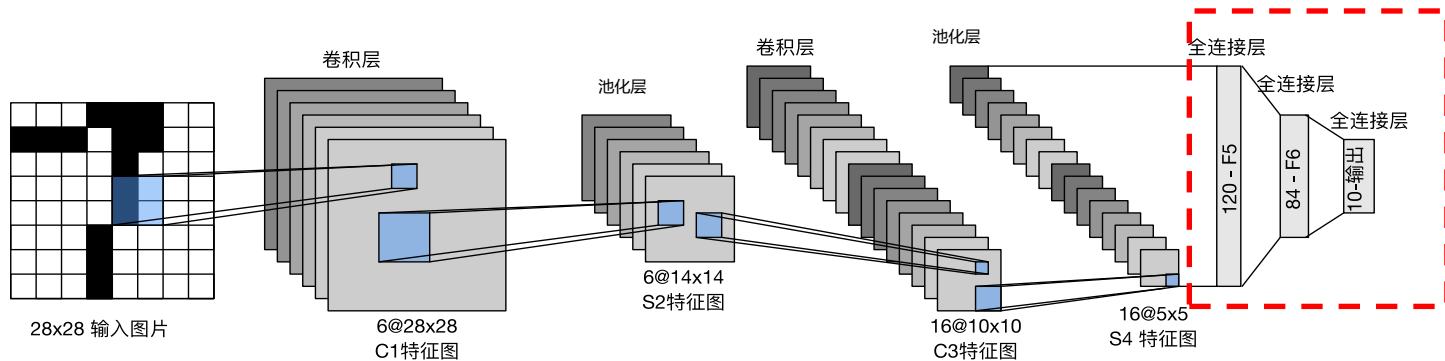
FC layer



Feedforward Neural Network

❖ Fully Connected Layer

- ❖ The fully connected layer's global input-related properties make it commonly used to integrate and transform features, as well as make final classification or regression predictions at the end of the network.



Feedforward Neural Network

❖ Demo

Training Stats

pause

Forward time per example: 1ms
Backprop time per example: 2ms
Classification loss: 1.58659
L2 Weight decay loss: 0.00094
Training accuracy: 0.44
Validation accuracy: 0.23636
Examples seen: 491
Learning rate: 0.01

change

Momentum: 0.9

change

Batch size: 20

change

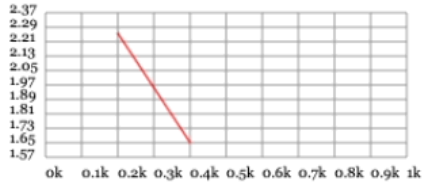
Weight decay: 0.001

change

save network snapshot as JSON

init network from JSON snapshot

Loss:



Examples	Loss
0k	2.30
0.1k	2.25
0.2k	2.20
0.3k	2.15
0.4k	2.10
0.5k	2.05
0.6k	2.00
0.7k	1.95
0.8k	1.90
0.9k	1.85
1k	1.80

clear graph

Instantiate a Network and Trainer

```
layer_defs = [];  
layer_defs.push({type:'input', out_sx:24, out_sy:24, out_depth:1});  
layer_defs.push({type:'conv', sx:5, filters:8, stride:1, pad:2, activation:'relu'});  
layer_defs.push({type:'pool', sx:2, stride:2});  
layer_defs.push({type:'conv', sx:5, filters:16, stride:1, pad:2, activation:'relu'});  
layer_defs.push({type:'pool', sx:3, stride:3});  
layer_defs.push({type:'softmax', num_classes:10});  
  
net = new convnetjs.Net();  
net.makeLayers(layer_defs);  
  
trainer = new convnetjs.SGDTrainer(net, {method:'adadelta', batch_size:20, l2_decay:0.001});
```

change network

Network Visualization

input (24x24x1)
max activation: 1, min: 0
max gradient: 0.08597, min: -0.10229

Activations:


Activation Gradients:


Network Visualization

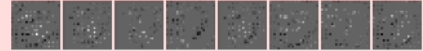
input (24x24x1)
max activation: 1, min: 0
max gradient: 0.00057, min: -0.00045

Activations:


Activation Gradients:


conv (24x24x8)
filter size 5x5x1, stride 1
max activation: 4.20194, min: -2.146
max gradient: 0.00026, min: -0.00017
parameters: 8x5x5x1+8 = 208

Activations:


Activation Gradients:


Weights:

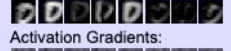
Weight Gradients:

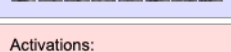

relu (24x24x8)
max activation: 4.20194, min: 0
max gradient: 0.00027, min: -0.00017

Activations:

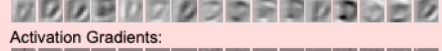

Activation Gradients:

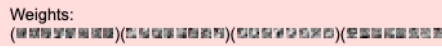

pool (12x12x8)
pooling size 2x2, stride 2
max activation: 4.20194, min: 0
max gradient: 0.00027, min: -0.00017

Activations:


Activation Gradients:


conv (12x12x16)
filter size 5x5x8, stride 1
max activation: 7.92743, min: -18.98162
max gradient: 0.00027, min: -0.00029
parameters: 16x5x5x8+16 = 3216

Activations:


Activation Gradients:


Weights:

Weight Gradients:




下面网页可体验卷积神经网络完成手写数字识别

Website:

<https://cs.stanford.edu/people/karpathy/convnetjs/demo/mnist.html>

Unsupervised learning

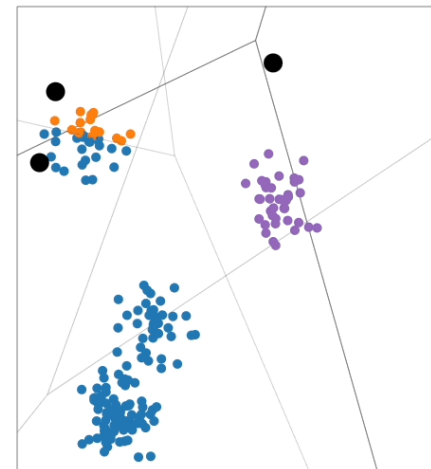
❖ K-means

K-means is the simplest clustering analysis algorithm, which divides the samples in the data set into K clusters, so that the sum of the distances between each sample and the center (i.e., centroid) of its cluster is minimized.

Enter the link below to practice:

Demo website:
<http://alekseynp.com/viz/k-means.html>

K-means Demonstration



Controls

Click and drag circles.

Data Generation

Method:

Points:

Clusters:

☒ constant data cluster size

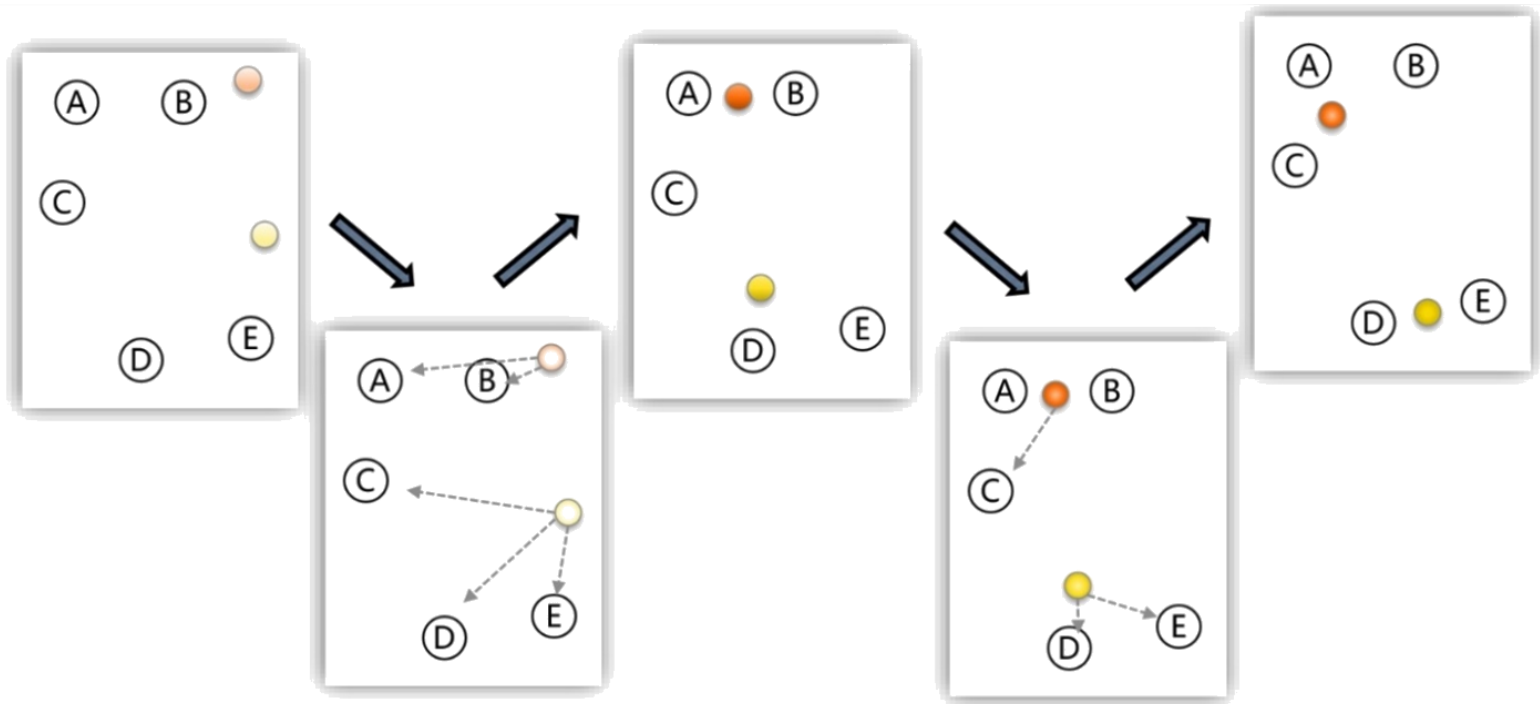
K-means

Clusters:

☐ show history

Unsupervised learning

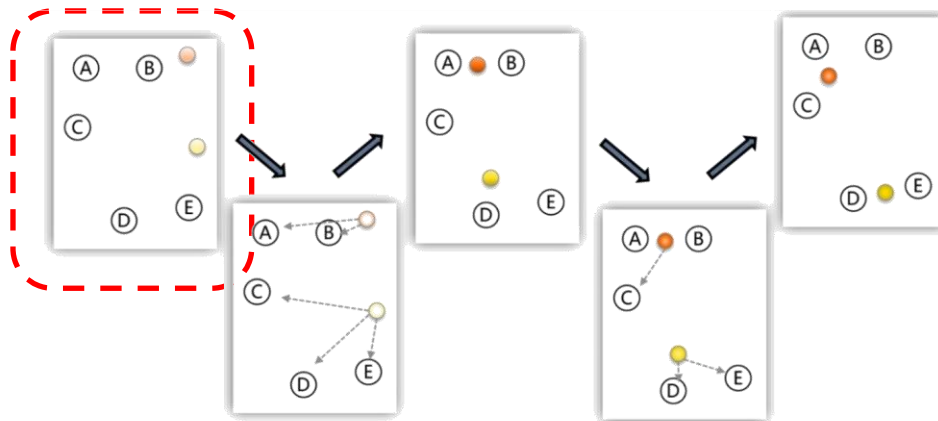
❖ K-means



Unsupervised learning

❖ K-means

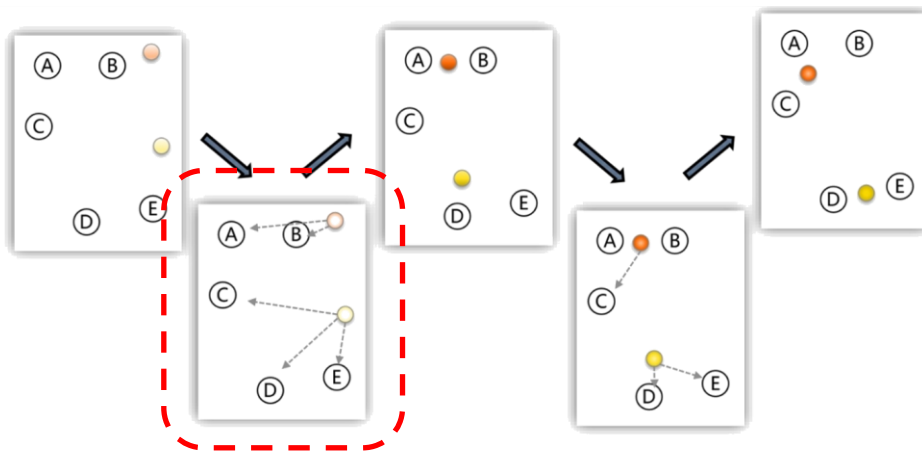
1) Select K value: Determine the number of clusters to be divided.
Here $K=2$, clustering 5 data points.



Unsupervised learning

❖ K-means

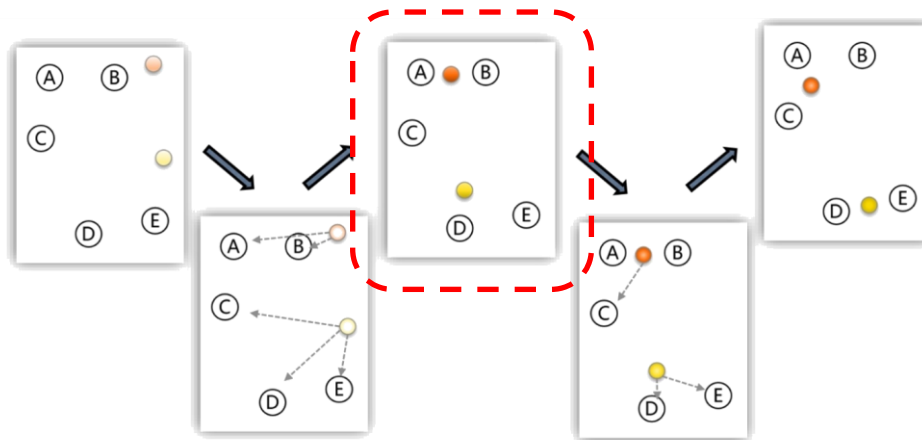
2) Initialize the centroid: There are two common ways to initialize the centroid. One is to randomly select a position for initialization, and the other is to randomly select a data point as the initial centroid.



Unsupervised learning

❖ K-means

3) Assign data points: Assign each data point to the cluster represented by the nearest centroid.



Unsupervised learning

❖ K-means

4) Update the centroid: Recalculate the centroid of each cluster, which is usually the mean of the coordinates of all points in the cluster.

质心更新公式:

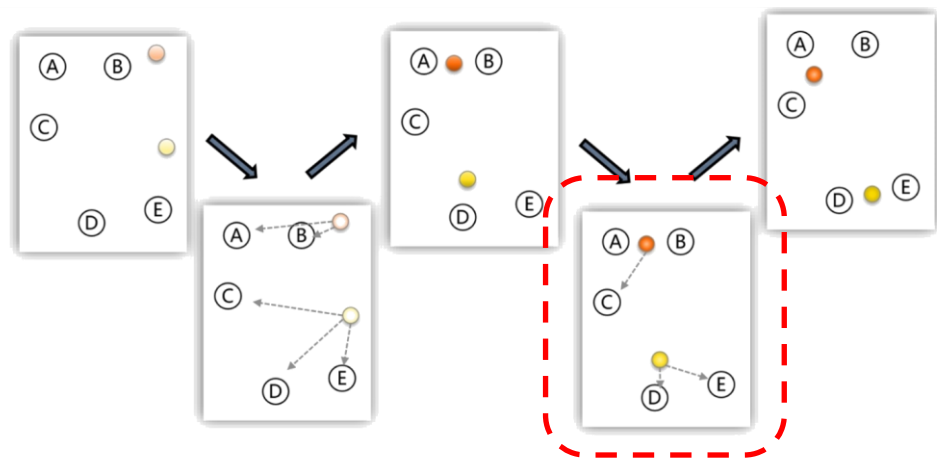
$$\mu_k = \frac{1}{|C_k|} \sum_{x \in C_k} x$$

其中:

μ_k 是第 k 个簇的质心。

$|C_k|$ 是簇 C_k 中数据点的数量 (即该簇中有多少个数据点)。

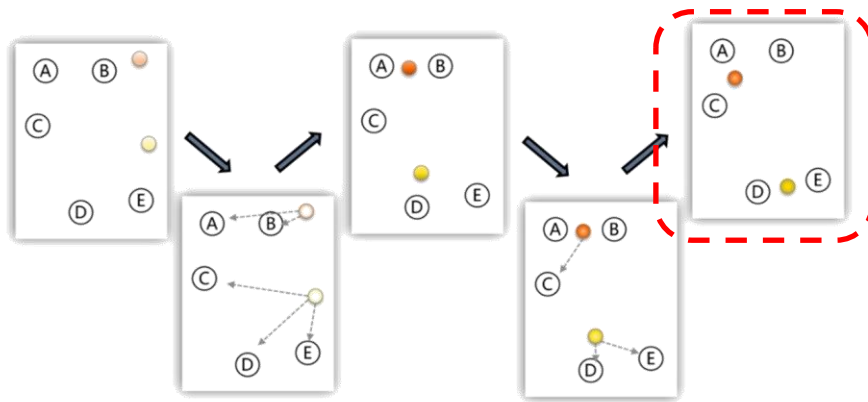
x 是簇 C_k 中的任意数据点坐标。



Unsupervised learning

❖ K-means

5) Iterative optimization: Repeat steps 3 and 4, i.e., assigning data points and updating the centroid, until any of the following conditions is met: the position of the centroid changes very little or no longer changes, or the preset number of iterations is reached.



Unsupervised learning

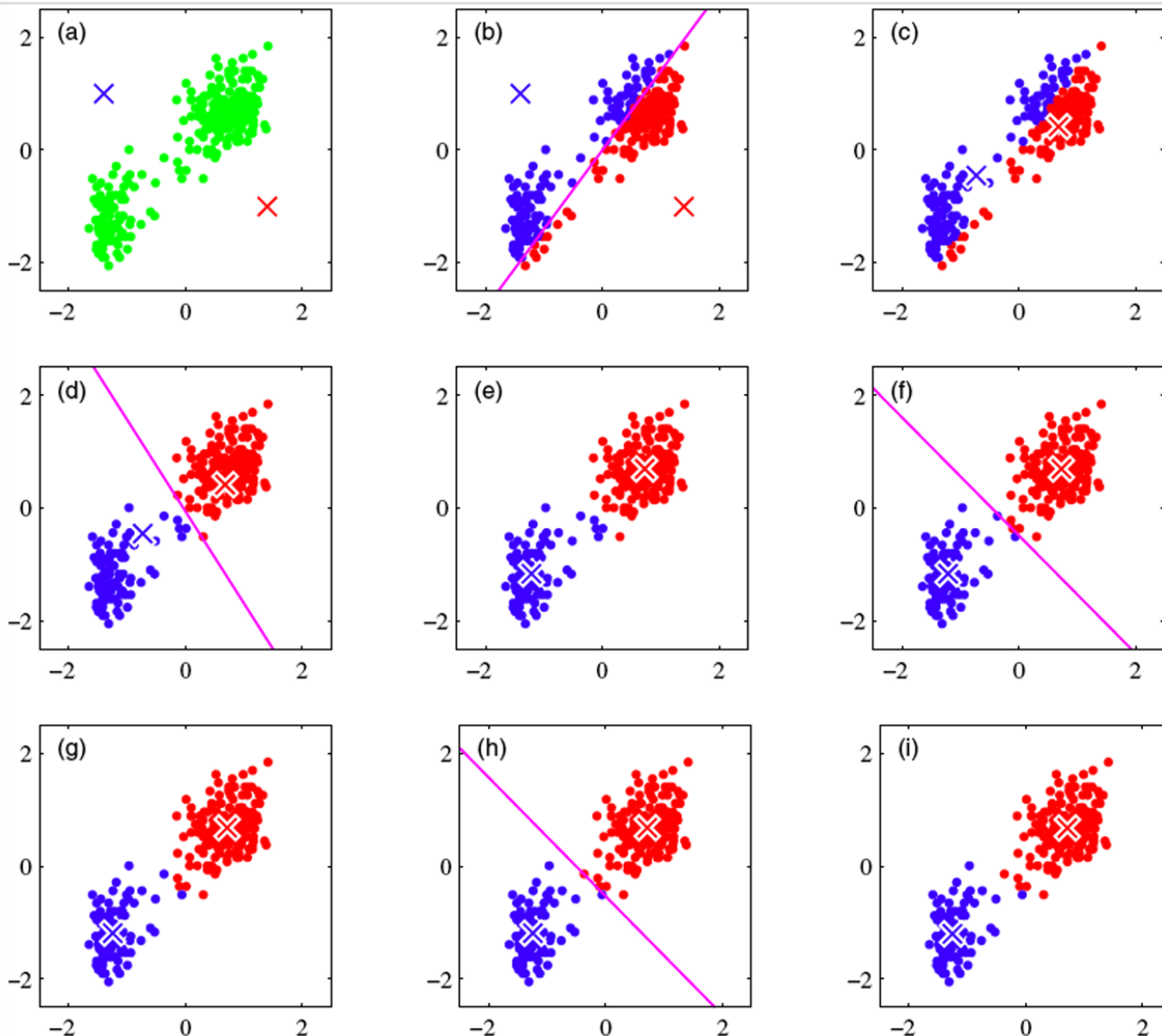


Figure from Bishop

Simple demo: <http://syskall.com/kmeans.js/>

End of Lecture

Can you understand artificial neural network and apply it to special applications?

LIUXIABI@BIT.EDU.CN

